

Contents

字串操作函式參考	4
函式一覽	4
contains	5
starts_with	6
ends_with	6
find	6
substring	6
char_at	6
replace	6
to_upper	7
to_lower	7
trim	7
trim_start	7
trim_end	7
repeat	8
reverse	8
byte_len	8
split	8
join	8
to_int	9
to_float	9
to_str	9
內建函式參考	9
函式一覽	9
print	11
Some	11
len	12
has_key	12
add	12
remove	12
range	13
exit	13
args	13
append	14
pop	14
reverse (串列)	14
slice	14
take	14
tap	15
filter	15
map	15
sort	15
sort!	16
reverse!	16
appended	16
append!	16
first	17
last	17
get (映射)	17
集合參考 (元組、串列、映射、集合)	17
元組	17

串列	18
映射	24
集合	26
契約式設計 (Design by Contract)	29
前置條件 (require)	29
後置條件 (ensure)	29
組合範例	29
結構體不變量 (invariant)	30
規則	30
控制流程參考	30
if / elif / else	30
while	31
for	31
break	33
continue	33
... (Ellipsis)	33
match	34
作用域規則	36
指令	37
語法	37
支援的目標	37
內建指令	37
注意事項	40
錯誤參考	40
錯誤格式	40
編譯錯誤一覽	40
執行時錯誤一覽	42
函式參考	42
函式定義語法	42
引數與回傳值的型別	43
遞迴	43
多載	43
Unit 型別函式	43
Lambda 函式	44
閉包	44
函式型別	45
高階函式	45
UFCS (統一函式呼叫語法)	45
運算子多載	46
I/O 函式參考手冊	47
函式一覽	47
使用範例	48
錯誤處理	48
備註	49
數學函式 (std.math)	49
概述	49
常數	49
絕對值	49

捨入	50
冪次與平方根	50
對數	50
指數	50
三角函式	50
反三角函式	51
雙參數函式	51
判定函式	51
運算子參考	51
優先順序表	51
算術運算子	52
比較運算子	52
邏輯運算子	53
位元運算子	53
三元條件運算子	54
範圍運算子	54
空值合併運算子 (??)	54
複合賦值運算子	54
遞增 / 遞減運算子	55
運算的型別規則	55
運算子多載	56
套件參考	56
概述	56
匯入語法	57
套件解析	57
標準函式庫 (std)	58
搜尋路徑的優先順序	58
RY_PATH 環境變數	58
限制	58
建立套件檔案	59
專案管理	59
ry init - 專案初始化	59
ry self-update - 自我更新	60
ry.toml 設定檔	60
正規表達式函式參考手冊	61
函式一覽	61
支援的模式語法	61
使用範例	62
結構體參考	64
概述	64
定義語法	64
建構子	64
欄位存取	64
欄位賦值	65
作為函式引數與回傳值使用	65
巢狀結構體	65
限制與錯誤	65
列舉型別 (enum)	66
測試功能	67

執行方式	67
語法	67
輸出格式	68
範例	69
模擬 (Mock)	69
參數化測試 (@each)	70
基於屬性的測試 (@property)	70
限制事項	70
型別參考	71
型別一覽	71
型別標註語法	71
可用型別名稱一覽	72
型別別名	72
字面量型別	73
範圍型別	73
none 關鍵字與 Option 型別簡寫	74
F-String (字串插值)	74
型別轉換 (as)	75
帶關聯資料的 enum (ADT)	75
泛型 enum	76
Error 型別	76
union 型別	77
型別規則 (運算時的型別轉換)	78
型別安全性限制	78

[English](#) | [日本語](#) | [繁體中文](#)

字串操作函式參考

針對字串 (str) 的操作函式一覽。所有函式皆可使用 UFCS 記法。

注意：所有字串操作皆支援 UTF-8。len()、char_at()、substring()、find() 和 reverse() 以 Unicode 碼位為單位操作，而非位元組。如需取得位元組長度，請使用 byte_len()。

函式一覽

搜尋與判定

函式	簽名	說明
contains	(str, str) → bool	回傳是否包含子字串
starts_with	(str, str) → bool	回傳是否以前綴開頭
ends_with	(str, str) → bool	回傳是否以後綴結尾
find	(str, str) → Option<int>	回傳子字串的字元位置 (未找到為 None)

擷取與轉換

函式	簽名	說明
substring	(str, int, int) → str	取得子字串 (字元索引)
char_at	(str, int) → str	取得指定位置的 UTF-8 字元
replace	(str, str, str) → str	全部取代子字串

大小寫

函式	簽名	說明
<code>to_upper</code>	<code>str → str</code>	轉換為 ASCII 大寫
<code>to_lower</code>	<code>str → str</code>	轉換為 ASCII 小寫

去除空白

函式	簽名	說明
<code>trim</code>	<code>str → str</code>	去除前後的空白
<code>trim_start</code>	<code>str → str</code>	去除開頭的空白
<code>trim_end</code>	<code>str → str</code>	去除結尾的空白

生成與加工

函式	簽名	說明
<code>repeat</code>	<code>(str, int) → str</code>	將字串重複 <code>n</code> 次
<code>reverse</code>	<code>str → str</code>	反轉字串 (UTF-8 感知)
<code>byte_len</code>	<code>str → int</code>	回傳字串的位元組長度

分割與連接

函式	簽名	說明
<code>split</code>	<code>(str, str) → List<str></code>	以分隔符號分割
<code>join</code>	<code>(List<str>, str) → str</code>	以分隔符號連接

型別轉換

函式	簽名	說明
<code>to_int</code>	<code>str → int</code>	將字串轉換為整數
<code>to_float</code>	<code>str → float</code>	將字串轉換為浮點數
<code>to_str</code>	<code>int/float/bool/str → str</code>	將值轉換為字串

contains

簽名: `contains(s: str, sub: str) -> bool`

回傳字串 `s` 中是否包含子字串 `sub`。

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))     # false (UFCS)
```

starts_with

簽名: `starts_with(s: str, prefix: str) -> bool`

回傳字串 `s` 是否以 `prefix` 開頭。

```
print(starts_with("hello", "hel"))    # true
print("hello".starts_with("world"))   # false (UFCS)
```

ends_with

簽名: `ends_with(s: str, suffix: str) -> bool`

回傳字串 `s` 是否以 `suffix` 結尾。

```
print(ends_with("hello", "llo"))      # true
print("hello".ends_with("world"))     # false (UFCS)
```

find

簽名: `find(s: str, sub: str) -> Option<int>`

回傳字串 `s` 中子字串 `sub` 首次出現的字元位置。未找到時回傳 `None`。

```
print(find("hello world", "world"))    # Some(6)
print(find("hello", "xyz"))            # None
print("abcdef".find("cd"))             # Some(2) (UFCS)
```

substring

簽名: `substring(s: str, start: int, end: int) -> str`

回傳字串 `s` 從 `start` 到 `end` (不含) 的子字串。索引為字元位置 (UTF-8 感知)。

```
print(substring("hello world", 0, 5))  # hello
print(substring("hello world", 6, 11)) # world
print("abcdef".substring(1, 4))        # bcd (UFCS)
```

char_at

簽名: `char_at(s: str, i: int) -> str`

回傳字串 `s` 第 `i` 個位置的 UTF-8 字元作為字串。

```
print(char_at("hello", 0))             # h
print("abc".char_at(2))                 # c (UFCS)
```

replace

簽名: `replace(s: str, old: str, new: str) -> str`

回傳將字串 `s` 中所有 `old` 替換為 `new` 後的新字串。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))               # bbbbbb
print("foo bar foo".replace("foo", "baz"))      # baz bar baz (UFCS)
```

to_upper

簽名: to_upper(s: str) -> str

回傳將 ASCII 小寫字母 (a-z) 轉換為大寫後的新字串。

```
print(to_upper("hello"))                      # HELLO
print("Hello World".to_upper())               # HELLO WORLD (UFCS)
```

to_lower

簽名: to_lower(s: str) -> str

回傳將 ASCII 大寫字母 (A-Z) 轉換為小寫後的新字串。

```
print(to_lower("HELLO"))                     # hello
print("Hello World".to_lower())              # hello world (UFCS)
```

trim

簽名: trim(s: str) -> str

回傳去除字串前後空白字元 (空格、定位字元、換行、回車) 後的新字串。

```
print(trim(" hello "))                       # hello
print(" hi ".trim())                        # hi (UFCS)
```

trim_start

簽名: trim_start(s: str) -> str

回傳去除字串開頭空白字元後的新字串。

```
print(trim_start(" hello "))                 # hello
print(" hi".trim_start())                   # hi (UFCS)
```

trim_end

簽名: trim_end(s: str) -> str

回傳去除字串結尾空白字元後的新字串。

```
print(trim_end(" hello "))                  # hello
print("hi ".trim_end())                    # hi (UFCS)
```

repeat

簽名: `repeat(s: str, n: int) -> str`

回傳將字串 `s` 重複 `n` 次後的新字串。

```
print(repeat("ab", 3))      # ababab
print("ha".repeat(3))      # hahaha (UFCS)
```

reverse

簽名: `reverse(s: str) -> str`

回傳字元順序反轉後的新字串 (UTF-8 感知)。

```
print(reverse("hello"))    # olleh
print("abc".reverse())     # cba (UFCS)
```

byte_len

簽名: `byte_len(s: str) -> int`

回傳字串 `s` 的位元組長度。與回傳 UTF-8 字元數的 `len()` 不同, `byte_len()` 回傳的是位元組數。

```
print(byte_len("hello"))   # 5
print(byte_len("あいう"))  # 9
print(len("あいう"))       # 3 (字元數)
```

split

簽名: `split(s: str, delim: str) -> List<str>`

以分隔符號 `delim` 分割字串 `s`, 回傳 `List<str>`。

```
@const
parts = split("a,b,c", ",")
print(parts[0])  # a
print(parts[1])  # b
print(parts[2])  # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

join

簽名: `join(xs: List<str>, sep: str) -> str`

以分隔符號 `sep` 連接字串串列的元素, 回傳結合後的字串。

```
@const
parts = ["a", "b", "c"]
```

```
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
```

to_int

簽名: to_int(s: str) -> int

將字串轉換為整數。

```
print(to_int("42"))          # 42
print(to_int("-7"))          # -7
print("123".to_int())        # 123 (UFCS)
```

to_float

簽名: to_float(s: str) -> float

將字串轉換為浮點數。

```
print(to_float("3.14"))      # 3.14
print("2.5".to_float())      # 2.5 (UFCS)
```

to_str

簽名: to_str(v: int | float | bool | str) -> str

將值轉換為字串。

型別	轉換格式
int	%ld
float	%g
bool	"true" / "false"
str	直接回傳

```
print(to_str(42))            # 42
print(to_str(3.14))          # 3.14
print(to_str(true))          # true
print(99.to_str())           # 99 (UFCS)
```

[English](#) | [日本語](#) | [繁體中文](#)

內建函式參考

函式一覽

核心

函式	說明
print(expr)	將值輸出到標準輸出
len(x)	回傳串列、映射、集合的元素數量，或字串的 UTF-8 字元數

函式	說明
<code>range(n) / range(start, end) / range(start, end, step)</code>	生成整數串列
<code>exit(code)</code>	以指定的結束碼終止程序
<code>args()</code>	以 <code>List<str></code> 回傳命令列引數

Option

函式	說明
<code>Some(expr)</code>	建構 <code>Option</code> 型別的有值變體

集合操作

函式	說明
<code>has_key(map, key)</code>	回傳映射中是否存在該鍵
<code>add(set, value)</code>	向集合新增元素（重複則忽略）
<code>remove(set, value)</code>	從集合刪除元素
<code>append(list, value) / append!(list, value)</code>	向串列末尾新增元素（就地修改）
<code>appended(list, value)</code>	傳回新增元素後的新串列（非破壞性）
<code>pop(list)</code>	移除並回傳串列的最後一個元素（ <code>Option<T></code> ）
<code>reverse(list)</code>	傳回反轉後的新串列（也適用於字串）
<code>reverse!(list)</code>	就地反轉串列（破壞性）
<code>slice(list, start, end)</code>	傳回從 <code>start</code> 到 <code>end</code> 的新子串列
<code>take(list, n)</code>	傳回包含前 <code>n</code> 個元素的新串列
<code>tap(list, fn)</code>	對每個元素呼叫 <code>fn</code> 以執行副作用，傳回原始串列
<code>filter(list, pred)</code>	傳回僅包含滿足述詞的元素的串列
<code>map(list, fn)</code>	傳回將每個元素轉換後的新串列
<code>sort(list) / sort(list, comp)</code>	傳回排序後的新串列（預設升序）
<code>sort!(list) / sort!(list, comp)</code>	就地排序串列（破壞性）
<code>insert(list, i, val)</code>	在索引 <code>i</code> 處插入元素
<code>remove_at(list, i)</code>	移除並回傳索引 <code>i</code> 處的元素
<code>items(map)</code>	回傳（鍵, 值）元組的串列
<code>remove(map, key)</code>	刪除指定鍵的條目
<code>get(map, key)</code>	回傳鍵的值（ <code>Option<V></code> ）
<code>get(map, key, default)</code>	回傳鍵的值，若不存在則回傳預設值
<code>union(set, set)</code>	回傳兩個集合的聯集
<code>intersection(set, set)</code>	回傳兩個集合的交集
<code>difference(set, set)</code>	回傳兩個集合的差集
<code>symmetric_difference(set, set)</code>	回傳兩個集合的對稱差
<code>is_subset(set, set)</code>	回傳第一個集合是否為第二個的子集
<code>is_superset(set, set)</code>	回傳第一個集合是否為第二個的超集

字串操作

函式	說明
<code>contains(s, sub)</code>	是否包含子字串
<code>starts_with(s, prefix)</code>	是否以前綴開頭
<code>ends_with(s, suffix)</code>	是否以後綴結尾

函式	說明
<code>find(s, sub)</code>	子字串的字元位置 (<code>Option<int></code>)
<code>byte_len(s)</code>	回傳字串的位元組長度
<code>substring(s, start, end)</code>	取得子字串
<code>char_at(s, i)</code>	取得指定位置的字元
<code>replace(s, old, new)</code>	全部取代子字串
<code>to_upper(s) / to_lower(s)</code>	大小寫轉換
<code>trim(s) / trim_start(s) / trim_end(s)</code>	去除空白
<code>repeat(s, n)</code>	將字串重複 <code>n</code> 次
<code>reverse(s)</code>	反轉字串
<code>split(s, delim)</code>	分割字串並回傳串列
<code>join(list, sep)</code>	以分隔符號連接串列中的字串
<code>to_int(s) / to_float(s) / to_str(v)</code>	型別轉換

→ 詳細請參閱 [字串操作函式參考](#)

print

簽名：`print(expr)`

將值輸出到標準輸出。末尾會加上換行。

型別	輸出格式
<code>int</code>	<code>%ld</code>
<code>float</code>	<code>%g</code>
<code>bool</code>	<code>true / false</code>
<code>str</code>	<code>%s</code>
<code>Option (Some)</code>	<code>Some(值)</code>
<code>Option (None)</code>	<code>None</code>
<code>list</code>	<code>[元素 1, 元素 2, ...]</code>
<code>map</code>	<code>{鍵 1: 值 1, 鍵 2: 值 2, ...}</code>
<code>set</code>	<code>{元素 1, 元素 2, ...}</code>
<code>enum</code>	變體名稱 (例如： <code>Red</code>)

```

print(42)           # 42
print(3.14)         # 3.14
print(true)         # true
print("hello")      # hello
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {a: 1}
print({1, 2, 3})    # {1, 2, 3}

```

錯誤條件：直接傳入結構體或元組會產生編譯錯誤。

Some

簽名：`Some(expr) -> Option<T>`

建構 `Option` 型別的有值變體。

```
@const
x: Option<int> = Some(42)
print(x)      # Some(42)
```

len

簽名: `len(x: List<T> | Map<K, V> | Set<T> | str) -> int`

回傳串列、映射、集合的元素數量，或字串的 UTF-8 字元數。如需取得位元組長度，請使用 `byte_len()`。

```
print(len([1, 2, 3]))      # 3
print(len({"a": 1, "b": 2})) # 2
print(len({1, 2, 3}))      # 3
print(len("hello"))        # 5
print(len("あいう"))       # 3 (UTF-8 字元數)
```

has_key

簽名: `has_key(m: Map<K, V>, key: K) -> bool`

回傳映射中是否存在指定的鍵。也可使用 UFCS 記法。

```
@const
m = {"a": 1, "b": 2}
print(has_key(m, "a"))    # true
print(m.has_key("z"))     # false (UFCS)
```

add

簽名: `add(s: Set<T>, value: T)`

向集合新增元素。若元素已存在則不做任何操作。也可使用 UFCS 記法。

```
@const
s = {1, 2, 3}
s.add(4)      # UFCS
add(s, 5)     # 一般呼叫
s.add(1)      # 已存在，因此忽略
print(len(s)) # 5
```

remove

簽名: `remove(s: Set<T>, value: T)`

從集合刪除元素。也可使用 UFCS 記法。

```
@const
s = {1, 2, 3}
s.remove(2)    # UFCS
print(2 in s)  # false
```

range

簽名：`range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

生成整數串列。

形式	生成的值
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (不包含 end)

- `step > 0` 時，從 `start` 向 `end` 遞增生成。
- `step < 0` 時，從 `start` 向 `end` 遞減生成。
- `step == 0` 時，會產生執行時錯誤。

```
print(range(3))           # [0, 1, 2]
print(range(2, 5))        # [2, 3, 4]
print(range(0, 10, 2))     # [0, 2, 4, 6, 8]
print(range(10, 0, -3))    # [10, 7, 4, 1]

for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

簽名：`exit(code: int)`

以指定的結束碼立即終止程序。`exit()` 之後的程式碼將不會被執行。

```
exit(0)    # 正常終止
exit(1)    # 錯誤終止
```

args

簽名：`args() -> List<str>`

以字串串列的形式回傳傳遞給腳本的命令列引數。不包含直譯器名稱或腳本檔案名稱——僅包含腳本路徑之後的引數。

```
# 執行: ry script.ry hello world
@const
a = args()
print(len(a))    # 2
print(a[0])      # hello
print(a[1])      # world

for x in args():
    print(x)
```

append

簽名：`append(list: List<T>, value: T)`

向串列末尾新增元素。此為就地修改操作——串列會被直接修改。也可使用 UFCS 記法。

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

簽名：`pop(list: List<T>) -> Option<T>`

移除並回傳串列的最後一個元素（`Option<T>`）。串列為空時回傳 `None`。也可使用 UFCS 記法。

```
xs = [1, 2, 3]
@const
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse (串列)

簽名：`reverse(list: List<T>) -> List<T>`

傳回元素順序反轉的新串列。原始串列不會被修改。也適用於字串（請參閱[字串操作](#)）。也可使用 UFCS 記法。

```
@const
xs = [1, 2, 3]
@const
ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (未修改)
```

slice

簽名：`slice(list: List<T>, start: int, end: int) -> List<T>`

傳回從 `start`（含）到 `end`（不含）的新子串列。索引會被鉗制在有效範圍內（0 到 `len(list)`）。也可使用 UFCS 記法。

```
@const
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (鉗制)
```

take

簽名：`take(list: List<T>, n: int) -> List<T>`

傳回包含前 `n` 個元素的新串列。若 `n` 超過串列長度，傳回整個串列的副本。若 `n <= 0`，傳回空串列。原始串列不會被修改。也可使用 UFCS 記法。

```

@const
xs = [1, 2, 3, 4, 5]
@const
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10)) # [1, 2, 3, 4, 5] (鉗制)
print(xs.take(0))  # []

```

tap

簽名: `tap(list: List<T>, fn: fn(T) -> R) -> List<T>`

對每個元素呼叫給定函式（忽略回傳值），然後傳回原始串列。適用於方法鏈中的除錯或插入副作用。也可使用 UFCS 記法。

```

@const
xs = [1, 2, 3]
@const
ys = xs.tap(fn(x: int): print(x)).map(fn(x: int): x * 2)
# 輸出 1, 2, 3, 然後 ys = [2, 4, 6]

```

filter

簽名: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

傳回僅包含述詞回傳 `true` 的元素的新串列。原始串列不會被修改。也可使用 UFCS 記法。

```

@const
xs = [1, 2, 3, 4, 5]
@const
ys = xs.filter(fn(x: int): x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (未修改)

```

map

簽名: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

傳回將每個元素以給定函式轉換後的新串列。輸出元素型別可以與輸入不同。原始串列不會被修改。也可使用 UFCS 記法。

```

@const
xs = [1, 2, 3]
@const
ys = xs.map(fn(x: int): x * 2)
print(ys)    # [2, 4, 6]

```

sort

簽名: `sort(list: List<T>) -> List<T> / sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

傳回排序後的新串列。預設為升序。可提供自訂比較函式（第一引數應排在第二引數之前時回傳 `true`）。原始串列不會被修改。排序是穩定的（相等元素保持原始順序）。也可使用 UFCS 記法。

```
@const
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降序排序
@const
desc = xs.sort(fn(a: int, b: int): a > b)
print(desc)        # [3, 2, 1]
```

sort!

簽名: `sort!(list: List<T>) / sort!(list: List<T>, comp: fn(T, T) -> bool)`

就地排序串列。排序演算法與 `sort()` 相同，但修改原始串列而非建立新串列。也可使用 UFCS 記法。

```
xs = [3, 1, 2]
xs.sort!()
print(xs)        # [1, 2, 3]
```

reverse!

簽名: `reverse!(list: List<T>)`

就地反轉串列。也可使用 UFCS 記法。

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)        # [3, 2, 1]
```

appended

簽名: `appended(list: List<T>, value: T) -> List<T>`

傳回新增元素後的新串列。原始串列不會被修改。也可使用 UFCS 記法。

```
@const
xs = [1, 2]
@const
ys = xs.appended(3)
print(xs)    # [1, 2] (未修改)
print(ys)    # [1, 2, 3]
```

append!

簽名: `append!(list: List<T>, value: T)`

`append()` 的別名。就地向串列末尾新增元素。為配合 `!` 命名慣例而提供。

first

簽名: `first(list: List<T>) -> Option<T>`

傳回串列的第一個元素 (`Option<T>`)。串列為空時回傳 `None`。

```
print(first([10, 20, 30]))    # Some(10)
```

last

簽名: `last(list: List<T>) -> Option<T>`

傳回串列的最後一個元素 (`Option<T>`)。串列為空時回傳 `None`。

```
print(last([10, 20, 30]))    # Some(30)
```

get (映射)

簽名: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

兩引數形式回傳鍵的值 (`Option<V>`)。三引數形式回傳鍵的值，若不存在則回傳預設值。

```
@const
m = {"a": 1, "b": 2}
print(get(m, "a"))           # Some(1)
print(get(m, "z"))           # None
print(get(m, "z", 0))        # 0
```

[English](#) | [日本語](#) | [繁體中文](#)

集合參考 (元組、串列、映射、集合)

元組

概述

固定長度、異質型別的值組合。以 LLVM literal StructType 實作，是堆疊上的值型別。

語法

```
@const
t = (1, 3.14)
@const
t: (int, float) = (1, 3.14)
```

型別標註

```
@const
pair: (int, str) = (42, "hello")
@const
triple: (int, float, bool) = (1, 2.0, true)
```

元素存取

使用 `.0`、`.1`、`...` 的數值索引存取。

```
@const
t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

函式回傳值

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
@const
result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

限制與錯誤

限制	詳細
超出範圍的索引	編譯錯誤
直接將元組傳給 <code>print</code>	編譯錯誤 (<code>print</code> 不支援)

串列

概述

相同型別的可變長度序列。分配於堆積上。

語法

```
@const
xs = [1, 2, 3]
@const
xs: List<int> = [1, 2, 3]
```

支援的元素型別

`int`, `float`, `bool`, `str`

索引存取

```
@const
xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

索引賦值

```
@const
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
```

len

```
@const
xs = [1, 2, 3]
print(len(xs))    # 3
```

print

```
@const
xs = [1, 2, 3]
print(xs)         # [1, 2, 3]
```

for 走訪

```
@const
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

向串列末尾新增元素。此為就地修改操作。

```
xs = [1, 2]
xs.append(3)
print(xs)         # [1, 2, 3]
```

pop

移除並回傳串列的最後一個元素。對空串列呼叫會產生執行時錯誤。

```
xs = [1, 2, 3]
@const
v = xs.pop()
print(v)          # 3
print(xs)         # [1, 2]
```

reverse

傳回元素順序反轉的新串列。原始串列不會被修改。也適用於字串。

```
@const
xs = [1, 2, 3]
print(reverse(xs)) # [3, 2, 1]
print(xs)          # [1, 2, 3] (未修改)
```

slice

傳回從 start（含）到 end（不含）的新子串列。索引會被鉗制在有效範圍內。

```
@const
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (鉗制)
```

take

傳回包含前 n 個元素的新串列。若 n 超過串列長度，傳回整個串列的副本。若 $n \leq 0$ ，傳回空串列。原始串列不會被修改。

```
@const
xs = [1, 2, 3, 4, 5]
@const
ys = xs.take(3)
print(ys)    # [1, 2, 3]
print(xs.take(10))  # [1, 2, 3, 4, 5] (鉗制)
print(xs.take(0))   # []
```

tap

對每個元素呼叫給定函式（忽略回傳值），然後傳回原始串列。適用於方法鏈中的除錯或插入副作用。

```
@const
xs = [1, 2, 3]
@const
ys = xs.tap(fn(x: int): print(x)).map(fn(x: int): x * 2)
# 輸出 1, 2, 3，然後 ys = [2, 4, 6]
```

filter

傳回僅包含滿足述詞的元素的新串列。原始串列不會被修改。

```
@const
xs = [1, 2, 3, 4, 5]
@const
ys = xs.filter(fn(x: int): x > 3)
print(ys)    # [4, 5]
```

map

傳回將每個元素以給定函式轉換後的新串列。輸出元素型別可以與輸入不同。原始串列不會被修改。

```
@const
xs = [1, 2, 3]
@const
ys = xs.map(fn(x: int): x * 2)
print(ys)    # [2, 4, 6]
```

sort

傳回排序後的新串列。預設為升序。可提供自訂比較函式。原始串列不會被修改。排序是穩定的（相等元素保持原始順序）。內部使用 TimSort。

```
@const
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降序排序
@const
desc = xs.sort(fn(a: int, b: int): a > b)
print(desc)    # [3, 2, 1]
```

filter、map、sort 的鏈接

這些函式傳回新串列，因此可透過 UFCS 進行鏈接。

```
@const
xs = [5, 3, 1, 4, 2]
@const
result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce

使用累加函式將串列歸約為單一值，以第一個元素作為初始值。

```
@const
xs = [1, 2, 3, 4, 5]
@const
total = reduce(xs, fn(a: int, b: int): a + b)
print(total)    # 15
```

fold

使用明確的初始值和累加函式將串列折疊為單一值。

```
@const
xs = [1, 2, 3, 4, 5]
@const
total = fold(xs, 0, fn(a: int, b: int): a + b)
print(total)    # 15
```

any

如果至少有一個元素滿足述詞，則傳回 true。

```
@const
xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int): x > 4))    # true
print(any(xs, fn(x: int): x > 9))    # false
```

all

如果所有元素都滿足述詞，則傳回 true。

```
@const
xs = [2, 4, 6]
print(all(xs, fn(x: int): x > 0))    # true
print(all(xs, fn(x: int): x > 3))    # false
```

sum

傳回所有元素的總和。

```
@const
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

傳回最小的元素。

```
@const
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

傳回最大的元素。

```
@const
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

傳回第一個元素。對空串列呼叫會產生執行時錯誤。

```
@const
xs = [10, 20, 30]
print(first(xs))   # 10
```

last

傳回最後一個元素。對空串列呼叫會產生執行時錯誤。

```
@const
xs = [10, 20, 30]
print(last(xs))    # 30
```

is_empty

如果串列沒有元素則傳回 true。

```
@const
xs = [1, 2, 3]
print(is_empty(xs)) # false
```

enumerate

傳回（索引，元素）元組的串列。

```
@const
xs = [10, 20, 30]
@const
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for 迴圈中的元組解構
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

將兩個串列合併為（元素 1，元素 2）元組的串列。結果長度等於較短的串列。

```

@const
xs = [1, 2, 3]
@const
ys = ["a", "b", "c"]
@const
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for 迴圈中的元組解構
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c

```

insert

在指定索引處插入元素。該索引及之後的元素向右移動。

```

xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]

```

remove_at

移除並回傳指定索引處的元素。該索引之後的元素向左移動。

```

xs = [1, 2, 3, 4]
@const
v = remove_at(xs, 1)
print(v)    # 2
print(xs)    # [1, 3, 4]

```

remove

從串列中移除第一個符合的指定值。若找不到該值，則不做任何操作。這是一個可變操作。

```

xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]

```

distinct

回傳一個移除重複元素的新串列。保持原始順序（保留第一次出現）。原始串列不會被修改。

```

@const
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)              # [1, 2, 3, 2, 1, 4] (未變更)

```

flatten

將巢狀串列（串列的串列）展開一層。回傳新串列。原始串列不會被修改。

```

@const
xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)              # [[1, 2], [3, 4]] (未變更)

```

操作複雜度

操作	複雜度
<code>xs[i]</code> 索引存取	$O(1)$
<code>append / append!</code>	均攤 $O(1)$
<code>pop</code>	$O(1)$
<code>first</code> 、 <code>last</code>	$O(1)$
<code>insert</code> 、 <code>remove_at</code>	$O(n)$
<code>sort / sort!</code>	$O(n \log n)$
<code>take</code>	$O(n)$
<code>tap</code>	$O(n)$
<code>filter</code> 、 <code>map</code> 、 <code>reduce</code> 、 <code>fold</code>	$O(n)$
<code>reverse / reverse!</code>	$O(n)$
<code>distinct</code>	$O(n)$
<code>len</code>	$O(1)$

限制與錯誤

限制	詳細
所有元素必須為相同型別	混合不同型別會產生編譯錯誤
空串列 <code>[]</code>	無法進行型別推論，會產生編譯錯誤
超出範圍的存取	執行時錯誤 (<code>exit(1)</code>)

映射

概述

鍵與值的對應表。分配於堆積上。

語法

```
@const
m = {"a": 1, "b": 2}
@const
m: Map<str, int> = {"a": 1, "b": 2}
```

鍵存取

```
@const
m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

插入與更新

```
@const
m = {"a": 1}
m["b"] = 2      # 新增
m["a"] = 99     # 更新
```

len

```
@const
m = {"a": 1, "b": 2, "c": 3}
print(len(m))   # 3
```

print

```
@const
m = {"a": 1, "b": 2}
print(m)    # {a: 1, b: 2}
```

has__key

```
@const
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

keys

傳回映射中所有鍵的串列。

```
@const
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
```

values

傳回映射中所有值的串列。

```
@const
m = {"a": 1, "b": 2, "c": 3}
print(values(m))    # [1, 2, 3]
```

items

回傳映射中所有條目的（鍵，值）元組串列。

```
@const
m = {"a": 1, "b": 2}
@const
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (映射)

從映射中刪除指定鍵的條目。若鍵不存在則不做任何操作。

```
@const
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {b: 2}
```

get

回傳指定鍵的值，若鍵不存在則回傳預設值。

```
@const
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

回傳一個合併兩個映射的新映射。當鍵重複時，第二個映射的值優先。原始映射不會被修改。

```
@const
m1 = {"a": 1, "b": 2}
@const
m2 = {"b": 99, "c": 3}
@const
m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

限制與錯誤

限制	詳細
所有鍵必須為相同型別	混合不同型別的鍵會產生編譯錯誤
所有值必須為相同型別	混合不同型別的值會產生編譯錯誤
空映射	需要型別標註（如 <code>m: Map<str, int> = {"a": 1}</code> ）
存取不存在的鍵	執行時錯誤（ <code>exit(1)</code> ）
鍵搜尋	雜湊表（平均 $O(1)$ ）
容量超過時	自動擴展為 2 倍

集合

概述

保持相同型別的元素且不重複的集合。分配於堆積上。

語法

```
@const
s = {1, 2, 3}
@const
s: Set<int> = {1, 2, 3}
```

支援的元素型別

int, float, bool, str

in 運算子（歸屬檢查）

```
@const
s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

len

```
@const
s = {1, 2, 3}
print(len(s))    # 3
```

print

```
@const
s = {1, 2, 3}
print(s)    # {1, 2, 3}
```

add (新增元素)

新增重複的元素時會被忽略。

```
@const
s = {1, 2, 3}
s.add(4)      # 新增
s.add(1)      # 已存在，因此忽略
print(len(s)) # 4
```

remove (刪除元素)

```
@const
s = {1, 2, 3}
s.remove(2)
print(2 in s) # false
```

for 走訪

```
@const
s = {10, 20, 30}
for x in s:
    print(x)
```

空集合

空集合需要型別標註。

```
@const
s: Set<int> = {}
```

函式引數

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

回傳包含兩個集合所有元素的新集合。

```
@const
a = {1, 2, 3}
@const
b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

回傳僅包含兩個集合中都存在的元素的新集合。

```
@const
a = {1, 2, 3}
@const
```

```
b = {2, 3, 4}
print(intersection(a, b))    # {2, 3}
```

difference

回傳包含在第一個集合中但不在第二個集合中的元素的新集合。

```
@const
a = {1, 2, 3}
@const
b = {2, 3, 4}
print(difference(a, b))    # {1}
```

symmetric_difference

回傳包含在任一集合中但不同時在兩個集合中的元素的新集合。

```
@const
a = {1, 2, 3}
@const
b = {2, 3, 4}
print(symetric_difference(a, b))    # {1, 4}
```

is_subset

如果第一個集合的所有元素都包含在第二個集合中，則回傳 true。

```
@const
a = {1, 2}
@const
b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

is_superset

如果第一個集合包含第二個集合的所有元素，則回傳 true。

```
@const
a = {1, 2, 3}
@const
b = {1, 2}
print(is_superset(a, b))    # true
print(is_superset(b, a))    # false
```

限制與錯誤

限制	詳細
所有元素必須為相同型別	混合不同型別會產生編譯錯誤
空集合 {}	需要型別標註
元素搜尋	雜湊表 (平均 O(1))
容量超過時	自動擴展為 2 倍

契約式設計 (Design by Contract)

Ry 支援 Eiffel 風格的契約式設計，包含前置條件 (require)、後置條件 (ensure) 和結構體不變量 (invariant)。契約違反時，程式將以 `exit(1)` 終止。

前置條件 (require)

前置條件在函式進入時檢查。它們指定函式被正確呼叫所需滿足的條件。

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance
```

若前置條件未滿足，程式將輸出以下訊息並終止：

```
Contract violation: require failed in deposit()
```

後置條件 (ensure)

後置條件在每個 `return` 之前檢查。它們指定函式對其回傳值的保證。

result 關鍵字

在 `ensure` 區塊中，`result` 指向回傳值。

```
fn abs(x: int) -> int:
    ensure:
        result >= 0
    if x < 0:
        return -x
    return x
```

old() 表達式

`old(expr)` 擷取函式本體執行前表達式的值。適用於比較前後狀態。

```
fn increment(x: int) -> int:
    ensure:
        result == old(x) + 1
    return x + 1
```

組合範例

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    ensure:
        result >= 0
        result == old(balance) + amount
```

```
new_balance: int = balance + amount
return new_balance
```

結構體不變量 (invariant)

不變量是結構體實例必須始終滿足的條件。在以下時機檢查：- 建構時 - 每次欄位賦值後

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

@const
a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1             # Contract violation: invariant failed
```

規則

- require 和 ensure 區塊為選用，寫在函式本體之前。
- 同時使用時，require 必須在 ensure 之前。
- result 和 old() 只能在 ensure 區塊中使用。
- invariant 寫在 record 定義的末尾，所有欄位宣告之後。
- 所有契約違反以 exit(1) 終止程式並輸出診斷訊息。

[English](#) | [日本語](#) | [繁體中文](#)

控制流程參考

if / elif / else

語法

```
if 條件式:
    # then 區塊
elif 條件式:
    # elif 區塊 (可多個)
else:
    # else 區塊 (可省略)
```

條件式的型別

型別	為 false 的值	為 true 的值
bool	false	true
int	0	非 0

float 和 str 無法直接用於條件式。

範例

```
@const
x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
else:
    print("small")
```

作用域規則

- if / elif / else 的各個區塊分別擁有獨立的區塊作用域。
- 在區塊內宣告的變數無法從區塊外存取。

```
if true:
    @const
    y = 42
# y 在此處無法存取
```

while

語法

```
while 條件式:
    # 迴圈主體
```

當條件式為 true 時，重複執行迴圈主體。

範例

```
i = 0
while i < 5:
    print(i)
    i += 1
```

搭配 break / continue

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

語法

```
# 串列 / 集合走訪
for x in iterable_expr:
    # 各元素依序賦值給 x

# range (從 0 開始)
```

```

for i in range(n):
    # i = 0, 1, ..., n-1

# range (指定起始與結束)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (指定步長)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...

```

映射鍵值走訪

```

for k, v in map_expr:
    # k 為鍵, v 為各條目的值

```

元組解構

走訪 2 元素元組的列表 (例如 `enumerate()` 或 `zip()` 的回傳值) 時, 可以解構為兩個變數。使用 `_` 捨棄值。

```

@const
xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)          # 11, 22

for _, x in enumerate(xs):
    print(x)              # 捨棄索引

```

範圍運算子 (..)

`..` 運算子建立包含兩端的整數範圍。1 .. 5 產生 [1, 2, 3, 4, 5]。

```

for i in 1 .. 5:
    print(i)              # 1 2 3 4 5

```

範例

```

@const
xs = [10, 20, 30]
for x in xs:
    print(x)

@const
s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)              # 0 1 2 3 4

for i in range(2, 6):
    print(i)              # 2 3 4 5

```

```

for i in range(0, 10, 2):
    print(i)      # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)      # 10 7 4 1

# 映射走訪
@const
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# 範圍運算子
for i in 1 .. 3:
    print(i)      # 1 2 3

```

break

- 立即跳出最內層的迴圈 (while 或 for)。
- 在迴圈外使用會產生編譯錯誤。

```

for i in range(10):
    if i == 5:
        break      # 在 i == 5 時跳出
    print(i)        # 0 1 2 3 4

```

錯誤範例

```

# 在迴圈外使用 break 會產生編譯錯誤
break      # Error: break outside loop

```

continue

- 結束最內層迴圈的當前迭代，跳至下一次迭代。
- 在迴圈外使用會產生編譯錯誤。

```

for i in range(5):
    if i == 2:
        continue    # 跳過 i == 2
    print(i)         # 0 1 3 4

```

... (Ellipsis)

- 不執行任何操作的語句 (no-op)。用作空區塊的佔位符。
- 可在任何區塊中使用：函數主體、if/elif/else、while、for、match case 等。

```

fn not_yet():
    ...

if true:

```

```
...
else:
    ...
```

match

語法

```
match 運算式:
    case 模式:
        # 主體
    case 模式 if 守衛條件:
        # 帶守衛的主體
    case _:
        # 萬用字元 (匹配任何值)
```

模式的種類

模式	範例	說明
萬用字元	<code>_</code>	匹配任何值
字面值	<code>0, "hello", true</code>	值的相等比較
變數綁定	<code>n</code>	匹配任何值並綁定到變數
enum 變體	<code>Color::Red</code>	enum 標籤的比較 (簡單 enum)
ADT enum 變體	<code>Shape::Circle(r)</code>	匹配帶有關聯資料的 enum 變體並綁定
<code>Some(x)</code>	<code>Some(v)</code>	當 Option 有值時，綁定其內容
<code>None</code>	<code>None</code>	當 Option 無值時
OR 模式	<code>1 \ 2 \ 3</code>	任一替代方案匹配時即匹配

guard 子句

可以使用 `case 模式 if 條件式:` 的形式指定守衛條件。只有當模式匹配且守衛條件為真時，該分支才會被執行。

OR 模式

可以使用 `|` 組合多個模式，任一模式匹配時即匹配。OR 模式中不允許使用變數綁定 (`n`、`Some(x)`、`Ok(v)`、`Err(e)`)。

```
match x:
    case 1 | 2 | 3:
        print("small")
    case _:
        print("other")

# enum OR 模式
match color:
    case Color::Red | Color::Blue:
        print("warm or cool")
    case Color::Green:
        print("green")
```

窮舉性檢查

- enum 型別：必須覆蓋所有變體或包含 `_`。OR 模式中的各替代方案會分別計算。
- Option 型別：必須覆蓋 Some 和 None 或包含 `_`。
- bool 型別：必須覆蓋 true 和 false 或包含 `_`。
- int / float / str 字面值：`_` 為必要。
- 帶守衛的分支不計入窮舉性。

範例

enum 匹配

```
enum Color:
```

```
    Red
```

```
    Green
```

```
    Blue
```

```
match color:
```

```
    case Color::Red:
```

```
        print("red")
```

```
    case Color::Green:
```

```
        print("green")
```

```
    case Color::Blue:
```

```
        print("blue")
```

Option 匹配

```
@const
```

```
x: Option<int> = Some(42)
```

```
match x:
```

```
    case Some(v):
```

```
        print(v)
```

```
    case None:
```

```
        print("nothing")
```

字面值匹配

```
match x:
```

```
    case 0:
```

```
        print("zero")
```

```
    case 1:
```

```
        print("one")
```

```
    case _:
```

```
        print("other")
```

guard 子句

```
match x:
```

```
    case n if n > 0:
```

```
        print("positive")
```

```
    case n if n < 0:
```

```
        print("negative")
```

```
    case _:
```

```
        print("zero")
```

ADT enum 匹配

當 enum 變體攜帶關聯資料時，使用綁定模式來取出值。

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

@const
s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)          # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")

```

多欄位變體會按宣告順序將各欄位綁定到不同的名稱。

作用域規則

- 各 case 分支擁有區塊作用域。
- 透過變數綁定模式 (n) 或 Some(x) 綁定的變數僅在該分支內有效。

作用域規則

區塊作用域

- if / elif / else / while / for / match 的各區塊擁有區塊作用域。
- 在區塊內宣告的變數會在區塊結束時離開作用域。

```

for i in range(3):
    @const
    tmp = i * 2
# tmp 在此處無法存取

if true:
    @const
    a = 1
# a 在此處無法存取

```

內層作用域的重新賦值

- 在內層作用域中對變數賦值會修改外層的變數（Python 風格的作用域）。
- 不會產生遮蔽——內層的賦值會修改同一個變數。

```

@const
x = 10
if true:
    x = 99 # 修改外層的 x
    print(x) # 99
print(x) # 99

```

[English](#) | [日本語](#) | [繁體中文](#)

指令

指令是可以附加到宣告上的編譯時元資料。使用 `@name` 語法。

語法

`@name`

`@name(key=value, ...)`

指令放置在目標宣告之前。可以堆疊多個指令。

支援的目標

指令可以套用到以下宣告：

- `fn` - 函式定義
- `record` - 結構體定義
- 變數宣告（使用 `@const` 或一般賦值）
- `record` 定義內的欄位
- `it` - 測試案例定義（僅限 `@each` 和 `@property`）

內建指令

`@deprecated`

將宣告標記為已棄用。當已棄用的實體被使用（呼叫、參照或存取）時，會發出編譯時警告。

套用於函式：

`@deprecated`

```
fn old_function() -> int:
    return 42
```

```
print(old_function())    # warning: 'old_function' is deprecated
```

套用於型別：

`@deprecated`

```
record OldPoint:
    x: int
    y: int
```

`@const`

```
p = OldPoint(1, 2)    # warning: 'OldPoint' is deprecated
```

套用於變數：

`@deprecated`

`@const`

```
old_value = 99
```

```
print(old_value)        # warning: 'old_value' is deprecated
```

套用於欄位：

`record Config:`

`@deprecated`

`old_setting: int`

`new_setting: int`

```
@const
c = Config(1, 2)
print(c.old_setting)      # warning: 'Config.old_setting' is deprecated
print(c.new_setting)      # 無警告
```

@const

將變數標記為不可變。使用 @const 宣告的變數在初始化後無法重新賦值。未使用 @const 時，變數預設為可變。

```
@const
x = 42
# x = 10    # 錯誤：無法重新賦值 @const 變數
```

搭配型別標註：

```
@const
name: str = "hello"
```

元組解構：

```
@const
a, b = (1, 2)
```

@native

宣告由執行環境（內建）提供實作的函式。該函式不能有函式本體。

基本語法：

```
@native
fn contains(s: str, sub: str) -> bool

print(contains("hello world", "world")) # true
```

運算子多載：

```
@native
fn operator+(a: str, b: str) -> str

print("hello" + " world") # hello world
```

與 UFCS 搭配使用：

```
@native
fn to_upper(s: str) -> str

print("hello".to_upper()) # HELLO
```

參數數量驗證：

當 @native 宣告包含型別簽名時，編譯器會在呼叫處驗證參數數量。支援重載函式（例如：1、2、3 個參數的 range），只要任一重載匹配即通過驗證。

標準函式庫宣告 (core/):

core/ 目錄包含所有內建函式的 @native 宣告，按類別組織：

檔案	內容
core/builtins.ry	print, len, range, enumerate, zip, exit, args

檔案	內容
core/str.ry	contains, starts_with, ends_with, find, substring, char_at, replace, to_upper, to_lower, trim, trim_start, trim_end, repeat, reverse, split, join
core/convert.ry	to_int, to_float, to_str
core/list.ry	append, pop, insert, remove_at, slice, distinct, flatten, sort, first, last, is_empty
core/map.ry	keys, values, items, has_key, get, merge
core/set.ry	add, remove, union, intersection, difference, symmetric_difference, is_subset, is_superset
core/higher_order.ry	filter, map, reduce, fold, any, all, sum, min, max

當 ry 執行檔附近存在 core/ 目錄時，這些檔案會作為前導自動載入。前導機制使得內建函式呼叫時的參數數量驗證生效。

限制事項: - @native 函式不能有本體（簽名後不能加:）。- 加上本體會導致解析錯誤: @native function must not have a body - 宣告的函式必須對應到現有的內建函式，否則在編譯時會發生錯誤。

未來擴充方向: - @native("libfoo.so") — 綁定外部共享函式庫的 FFI。

@each

啟用參數化測試，以不同參數多次執行 it 區塊。

語法:

```
@each([(引數 1, 引數 2, ...), ...])
it("描述 {0} 和 {1}", fn(param1: 型別, param2: 型別):
    # 測試主體
)
```

支援的目標: 僅限 it 呼叫

限制事項: - 引數必須是元組列表 - 元組的元素數量必須與 lambda 參數數量匹配 - 描述字串中的 {0}, {1}, ... 會被替換為參數的字串表示

@property

啟用基於屬性的測試，為 it 區塊生成隨機輸入。

語法:

```
@property(count=100)
it("屬性名稱", fn(a: int, b: int):
    # 使用隨機值的測試主體
)
```

支援的目標: 僅限 it 呼叫

參數:

參數	型別	預設值	說明
count	int	100	隨機試驗次數

支援的參數型別:

型別	範圍
int	-1000 到 1000
float	-1000.0 到 1000.0
bool	true 或 false
str	隨機 ASCII、0-20 字元

失敗時會顯示反例（導致失敗的參數值）。

參數（未來擴充）

指令支援可選的參數語法，為未來擴充做準備：

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

目前，參數會被解析但不會被 @deprecated 指令使用。

注意事項

- 已棄用的實體仍然正常運作，僅會發出警告。
- 警告在使用點發出，不在定義點發出。
- 定義已棄用的實體但不使用它，不會產生警告。
- 未知的指令名稱會導致解析錯誤。
- 在不支援的目標（如 if、while）上使用指令會導致解析錯誤。

[English](#) | [日本語](#) | [繁體中文](#)

錯誤參考

錯誤格式

ry 以 Rust 風格的豐富格式顯示編譯錯誤，顯示錯誤的確切位置及原始碼上下文：

```
error: cannot reassign @const variable: x
--> main.ry:5:1
|
5 | x = 10
|   ^ cannot reassign @const variable: x
```

每條錯誤訊息包含：- 錯誤等級和訊息 (error: ...) - 檔案位置 (--> 檔案: 行: 列) - 原始碼行 (相關程式碼) - 插入符號指示器 (^) 指向確切的列位置

編譯錯誤一覽

錯誤內容	原因	範例
使用未宣告的變數	參照了未宣告的變數	print(x) (x 未宣告)
對 @const 變數重新賦值	對以 @const 宣告的變數重新賦值	@const x = 1 → x = 2
重新宣告同名變數	在同一作用域重新宣告同名變數	x = 1 → 同名再宣告
變數型別變更賦值	對變數賦值不同型別的值	x = 1 → x = 3.14
型別標註不一致	宣告時的型別與賦值的型別不同	x: int = 3.14

錯誤內容	原因	範例
多載回傳型別重複	定義了引數型別相同但僅回傳型別不同的多載	引數 (int, int) 但回傳型別分別為 int 和 float 的兩個函式
多載解析失敗	不存在與引數型別匹配的多載	fn add(a: int, b: int) 卻呼叫 add(1.0, 2.0)
對位元運算傳入 float	對 &、\ 、^、~、<<、>> 傳入 float 型別	3.14 & 1
空串列	無法從空串列字面值 [] 推論型別	xs = []
空映射	無法從空映射字面值 {} 推論型別	m = {}
元組超出範圍的索引 在迴圈外使用 break/continue	存取元組中不存在的索引 在 for/while 迴圈外使用 break 或 continue	t = (1, 2) → t.2 在函式頂層使用 break
在區塊內匯入模組	在函式或條件區塊內使用 from 陳述式	在函式內使用 from math
循環匯入	模組之間互相匯入	a.ry 匯入 b.ry, b.ry 匯入 a.ry
相同欄位名重複	在結構體內定義了兩次相同的欄位名	在 type T: x: int 中定義了兩個 x
match 窮舉性不足	match 未覆蓋所有模式	enum 的部分變體未覆蓋、Option 缺少 None、字面值缺少 _
!! 回傳型別不匹配	在未回傳 (T, Error?) 的函式中使用 !!	在回傳 int 的函式中使用 !!
!! 運算元型別不匹配	對非 (T, Error?) 值使用 !!	對普通 int 使用 !!

編譯錯誤範例

```

# 對 @const 變數重新賦值
@const
x = 10
x = 20    # 錯誤

# 型別變更賦值
@const
n = 1
n = "hello"    # 錯誤：對 int 型別賦值 str 型別

# 空串列
@const
xs = []    # 錯誤：無法推論型別

# 在迴圈外使用 break
break    # 錯誤：在迴圈外

# 在區塊內匯入
fn foo():
    from math    # 錯誤：僅能在頂層匯入

```

```
# 相同欄位名重複
record Bad:
  x: int
  x: float # 錯誤：x 重複
```

執行時錯誤一覽

錯誤內容	原因	範例
串列超出範圍的存取	串列的索引超出範圍	<code>xs = [1, 2, 3] → xs[5]</code>
映射不存在的鍵存取	參照映射中不存在的鍵	<code>m = {"a": 1} → m["b"]</code>

契約違反 | `require`、`ensure` 或 `invariant` 條件評估為 `false` | 參閱[契約式設計](#) |

所有執行時錯誤都會以 `exit(1)` 結束程序。

執行時錯誤範例

```
# 串列超出範圍的存取
@const
xs = [1, 2, 3]
print(xs[10]) # 執行時錯誤：exit(1)
```

```
# 映射不存在的鍵存取
@const
m = {"a": 1}
print(m["z"]) # 執行時錯誤：exit(1)
```

[English](#) | [日本語](#) | [繁體中文](#)

函式參考

函式定義語法

`fn` 函式名 (引數名: 型別, ...) -> 回傳型別:

```
# 主體
return 值
```

- 引數型別為必要。
- 回傳型別可省略 (省略時為 `Unit`)。
- 主體為縮排的區塊。
- 函式可以定義 `require` (前置條件) 和 `ensure` (後置條件)。參閱 [契約式設計](#)。

命名慣例：函式名稱和引數名稱必須使用 `snake_case` (如 `add`、`get_value`、`map_list`)。編譯器會強制執行此慣例。

```
fn add(a: int, b: int) -> int:
  return a + b
```

```
fn greet(name: str):
  print("Hello, " + name) # 回傳型別為 Unit
```

引數與回傳值的型別

項目	說明
引數型別	必要。所有引數都需要型別標註
回傳型別	可省略。省略時為 <code>Unit</code> （相當於 <code>void</code> ）
<code>Unit</code>	不回傳值的函式的回傳型別

```
fn no_return(x: int):      # 回傳型別 Unit（省略）
    print(x)
```

```
fn get_value() -> int:    # 回傳型別 int
    return 42
```

遞迴

函式可以呼叫自身。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

多載

可以定義引數數量或型別不同的同名函式。

規則

- 引數的數量或型別不同即可定義同名函式。
- 呼叫時會根據引數的型別和數量選擇適當的函式。
- 僅回傳型別不同的多載是不允許的。

```
fn area(side: int) -> int:
    return side * side
```

```
fn area(w: int, h: int) -> int:
    return w * h
```

```
@const
a = area(5)      # 25
@const
b = area(3, 4)   # 12
```

Unit 型別函式

不回傳值的函式會回傳 `Unit`。回傳型別可以省略。

```
fn log(msg: str):
    print(msg)
```

```
fn log_typed(msg: str) -> Unit:
    print(msg)
```

Lambda 函式

可以就地定義匿名函式。

語法

單一運算式（運算式的值作為回傳值。回傳型別自動推論）
fn(引數名: 型別, ...): 運算式

多行區塊
fn(引數名: 型別, ...):
 # 多個陳述式
 return 值

明確指定回傳型別（可省略）
fn(引數名: 型別, ...) -> 回傳型別: 運算式

範例

```
@const
double = fn(x: int): x * 2
@const
result = double(5)    # 10

@const
add = fn(a: int, b: int): a + b
@const
sum = add(3, 4)       # 7

# 多行 lambda
@const
abs = fn(x: int):
    if x < 0:
        return -x
    return x
```

閉包

Lambda 函式會以值捕獲定義時外層作用域的變數。

```
base = 10
@const
add_base = fn(x: int): x + base    # 以值捕獲 base

base = 99                          # 不影響已捕獲的值
@const
r = add_base(5)    # 15（使用捕獲時的 base = 10）
```

捕獲規則

項目	內容
捕獲方式	值捕獲（複製）
捕獲時機	Lambda 定義時
外層變數修改的影響	無（因為是複製）

函式型別

用於將函式作為值處理的型別。

語法

`fn(引數型別 1, 引數型別 2, ...) -> 回傳型別`

範例

```
@const
f: fn(int) -> int = fn(x: int): x * 2
@const
g: fn(int, int) -> int = fn(a: int, b: int): a + b

fn apply(func: fn(int) -> int, x: int) -> int:
    return func(x)

@const
result = apply(f, 5)    # 10
```

高階函式

可以接收函式作為引數，或將函式作為回傳值回傳。

```
fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

@const
doubled = map_list([1, 2, 3], fn(x: int): x * 2)
# [2, 4, 6]
```

UFCS（統一函式呼叫語法）

可以使用 `a.f(b)` 的形式呼叫 `f(a, b)`。方便用於方法鏈。

語法

```
# 一般呼叫
f(a, b)
```

```
# UFCS 呼叫 (等價)
a.f(b)
```

鏈接

```
fn double(x: int) -> int:
    return x * 2

fn add_one(x: int) -> int:
    return x + 1

@const
result = 5.double().add_one()  # double(5) → 10, add_one(10) → 11
```

與欄位存取混用

欄位存取 (`.field`) 和 UFCS (`.method()`) 使用相同的點記法，但透過是否有引數來區分。

```
@const
p = Point(3, 4)
@const
len = p.x.to_float()  # 欄位存取 + UFCS
```

運算子多載

可以為使用者定義型別定義運算子的行為。

語法

```
# 二元運算子 (2 個引數)
fn operator<op>(a: 型別, b: 型別) -> 回傳型別:
    ...

# 一元運算子 (1 個引數)
fn operator<op>(a: 型別) -> 回傳型別:
    ...
```

可多載的運算子

種類	運算子
算術 (二元)	+ - * / % ** //
比較 (二元)	== != < <= > >=
位元 (二元)	& \ ^ << >>
邏輯 (二元)	and or
一元	- ~ not

二元 / 一元的區別

依引數個數區分。

```
record Vec2:
    x: float
    y: float
```

```

# 二元 +
fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

# 一元 -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

# 比較
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

@const
v1 = Vec2(1.0, 2.0)
@const
v2 = Vec2(3.0, 4.0)
@const
v3 = v1 + v2    # Vec2(4.0, 6.0)
@const
v4 = -v1        # Vec2(-1.0, -2.0)

```

[English](#) | [日本語](#) | [繁體中文](#)

I/O 函式參考手冊

標準輸入輸出與檔案操作的函式一覽。所有函式皆需從 `std.io` 明確匯入。

```
from std.io import read_text, write_text, file_exists
```

函式一覽

標準輸入

函式	簽名	說明
<code>read_line</code>	<code>() -> str</code>	從 <code>stdin</code> 讀取一行（移除末尾換行）
<code>read_all</code>	<code>() -> str</code>	讀取 <code>stdin</code> 直到 EOF

檔案 I/O

函式	簽名	說明
<code>read_text</code>	<code>(str) -> str</code>	將整個檔案作為字串讀取
<code>write_text</code>	<code>(str, str) -> Unit</code>	將字串寫入檔案（覆蓋）
<code>append_text</code>	<code>(str, str) -> Unit</code>	在檔案末尾追加字串
<code>file_exists</code>	<code>(str) -> bool</code>	檢查檔案是否存在
<code>delete_file</code>	<code>(str) -> Unit</code>	刪除檔案
<code>read_bytes</code>	<code>(str) -> List<byte></code>	將檔案作為位元組串列讀取
<code>write_bytes</code>	<code>(str, List<byte>) -> Unit</code>	將位元組串列寫入檔案

位元組轉換

函式	簽名	說明
<code>str_to_bytes</code>	<code>(str) -> List<byte></code>	將字串轉換為 UTF-8 位元組
<code>bytes_to_str</code>	<code>(List<byte>) -> str</code>	將位元組串列轉換為字串

使用範例

讀寫檔案

```
from std.io import read_text, write_text, append_text, file_exists, delete_file

write_text("hello.txt", "Hello, World!")
@const
content = read_text("hello.txt")
print(content)    # Hello, World!

append_text("hello.txt", "\nGoodbye!")
print(read_text("hello.txt"))
# Hello, World!
# Goodbye!

print(file_exists("hello.txt"))    # true
delete_file("hello.txt")
print(file_exists("hello.txt"))    # false
```

位元組操作

```
from std.io import str_to_bytes, bytes_to_str, write_bytes, read_bytes

@const
bs = str_to_bytes("ABC")
print(len(bs))    # 3

write_bytes("data.bin", bs)
@const
rb = read_bytes("data.bin")
@const
s = bytes_to_str(rb)
print(s)          # ABC
```

從標準輸入讀取

```
from std.io import read_line

@const
name = read_line()
print(f"Hello, {name}!")
```

錯誤處理

所有檔案操作在失敗時會以執行時錯誤終止：

操作	錯誤條件
<code>read_text</code> / <code>read_bytes</code>	檔案不存在或無法開啟

操作	錯誤條件
<code>write_text</code> / <code>write_bytes</code> / <code>append_text</code>	無法開啟檔案進行寫入
<code>delete_file</code>	無法刪除檔案

錯誤訊息會輸出到 `stderr`，程式以結束代碼 1 終止。

備註

- 使用 `List<byte>` 作為緩衝區型別。標準串列操作 (`len()`、`append()`、`slice()`、索引存取) 均可用於位元組串列。
- 檔案路徑若未指定絕對路徑，則為相對於當前工作目錄的相對路徑。
- `write_text` 與 `write_bytes` 會覆蓋既有檔案。若要追加內容，請使用 `append_text`。

[English](#) | [日本語](#) | [繁體中文](#)

數學函式 (`std.math`)

概述

`std.math` 套件提供數學常數與函式。與 `std` 套件不同，它不會自動匯入。需要使用明確匯入來存取函式。

```
from std.math import sqrt, PI, sin
```

常數

常數	型別	說明
<code>PI</code>	<code>float</code>	圓周率 (3.141592653589793)
<code>E</code>	<code>float</code>	尤拉數 (2.718281828459045)
<code>Inf</code>	<code>float</code>	正無窮大
<code>NaN</code>	<code>float</code>	非數值 (Not a Number)

```
from std.math import PI, E, Inf, NaN
```

```
@const
```

```
circumference = 2.0 * PI * radius
```

絕對值

函式	簽章	說明
<code>abs(x)</code>	<code>(int) -> int</code>	整數的絕對值
<code>abs(x)</code>	<code>(float) -> float</code>	浮點數的絕對值

```
from std.math import abs
```

```
abs(-5)      # 5
abs(-3.14)   # 3.14
```

捨入

函式	簽章	說明
<code>floor(x)</code>	<code>(float) -> int</code>	向下取整
<code>ceil(x)</code>	<code>(float) -> int</code>	向上取整
<code>round(x)</code>	<code>(float) -> int</code>	四捨五入

```
from std.math import floor, ceil, round
```

```
floor(3.7)    # 3
ceil(3.2)     # 4
round(3.5)    # 4
```

冪次與平方根

函式	簽章	說明
<code>sqrt(x)</code>	<code>(float) -> float</code>	平方根
<code>pow(x, y)</code>	<code>(float, float) -> float</code>	x 的 y 次方

```
from std.math import sqrt, pow
```

```
sqrt(9.0)     # 3.0
pow(2.0, 3.0)  # 8.0
```

對數

函式	簽章	說明
<code>log(x)</code>	<code>(float) -> float</code>	自然對數 (底數 e)
<code>log2(x)</code>	<code>(float) -> float</code>	底數 2 的對數
<code>log10(x)</code>	<code>(float) -> float</code>	常用對數 (底數 10)

指數

函式	簽章	說明
<code>exp(x)</code>	<code>(float) -> float</code>	e 的 x 次方

三角函式

函式	簽章	說明
<code>sin(x)</code>	<code>(float) -> float</code>	正弦 (x 為弧度)
<code>cos(x)</code>	<code>(float) -> float</code>	餘弦 (x 為弧度)

函式	簽章	說明
<code>tan(x)</code>	<code>(float) -> float</code>	正切 (x 為弧度)

反三角函式

函式	簽章	說明
<code>asin(x)</code>	<code>(float) -> float</code>	反正弦 (結果為弧度)
<code>acos(x)</code>	<code>(float) -> float</code>	反餘弦 (結果為弧度)
<code>atan(x)</code>	<code>(float) -> float</code>	反正切 (結果為弧度)

雙參數函式

函式	簽章	說明
<code>atan2(y, x)</code>	<code>(float, float) -> float</code>	y/x 的反正切 (結果為弧度)
<code>hypot(x, y)</code>	<code>(float, float) -> float</code>	斜邊長度: $\sqrt{x^2 + y^2}$

判定函式

函式	簽章	說明
<code>is_nan(x)</code>	<code>(float) -> bool</code>	若 x 為 NaN 則回傳 true
<code>is_inf(x)</code>	<code>(float) -> bool</code>	若 x 為正或負無窮大則回傳 true

```
from std.math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#) | [日本語](#) | [繁體中文](#)

運算子參考

優先順序表

優先順序數字越小越高（越先被求值）。

優先順序	運算子	說明	結合性
0	<code>!!</code>	錯誤傳播（後綴）	左
1	<code>()</code>	分組	—
2	<code>+x -x ~x</code>	一元正號、負號、位元 NOT	右
3	<code>**</code>	次方	右結合
3.5	<code>as</code>	型別轉換	左

優先順序	運算子	說明	結合性
4	* / % //	乘法、除法、餘數、整數除法	左
5	+ -	加法、減法	左
6	<< >> >>>	位元移位	左
7	&	位元 AND	左
8	^	位元 XOR	左
9	\	位元 OR	左
10	== != < <= > >= in not in	比較、歸屬	左
11	not	邏輯 NOT	右
12	and	邏輯 AND	左
13	or	邏輯 OR	左
13.5	??	空值合併	左
14	?:	三元條件	右

算術運算子

運算子	說明	範例
+	加法 / 字串串接	1 + 2 → 3、"a" + "b" → "ab"
-	減法	5 - 3 → 2
*	乘法 / 字串重複	4 * 3 → 12、"ab" * 3 → "ababab"
/	除法（始終為 float）	7 / 2 → 3.5
//	整數除法（截斷）	7 // 2 → 3
%	餘數	7 % 3 → 1
**	次方（始終為 float）	2 ** 10 → 1024.0
-x	一元負號	-5, -3.14
+x	一元正號	+5（不改變正負號）

```
@const
a = 10 // 3    # 3 (int)
@const
b = 10 / 3     # 3.3333... (float)
@const
c = 2 ** 8     # 256.0 (float)
@const
s = "foo" + "bar" # "foobar"
```

比較運算子

全部回傳 bool。

運算子	說明
==	等於
!=	不等於
<	小於
<=	小於等於
>	大於
>=	大於等於

- 可用於數值型別（int / float）和 bool。
- str 之間以字典序（位元組順序）比較。
- in 運算子用於集合、串列、映射的歸屬檢查（x in s）。

- `not in` 運算子為 `in` 的否定 (`x not in s`)。
- 對於映射，`in` 檢查鍵是否存在。

```
@const
x = 3 < 5          # true
@const
y = "abc" < "abd"  # true (字典序)
@const
s = {1, 2, 3}
@const
z = 2 in s         # true
@const
w = 4 not in s     # true
@const
xs = [1, 2, 3]
@const
a = 2 in xs        # true (串列線性搜尋)
@const
m = {"a": 1}
@const
b = "a" in m       # true (映射鍵搜尋)
```

邏輯運算子

運算子	說明	型別
<code>and</code>	邏輯 AND	<code>bool × bool → bool</code>
<code>or</code>	邏輯 OR	<code>bool × bool → bool</code>
<code>not</code>	邏輯 NOT	<code>bool → bool</code>

```
@const
a = true and false # false
@const
b = true or false  # true
@const
c = not true       # false
```

位元運算子

僅可用於 `int` 型別。對 `float` 或 `bool` 使用會產生編譯錯誤。

運算子	說明	範例
<code>&</code>	位元 AND	<code>0b1100 & 0b1010 → 0b1000</code>
<code>\ </code>	位元 OR	<code>0b1100 \ 0b1010 → 0b1110</code>
<code>^</code>	位元 XOR	<code>0b1100 ^ 0b1010 → 0b0110</code>
<code>~</code>	位元 NOT (一元)	<code>~0 → -1</code>
<code><<</code>	左移	<code>1 << 4 → 16</code>
<code>>></code>	算術右移	<code>16 >> 2 → 4</code>
<code>>>></code>	邏輯右移	<code>-1 >>> 1 → 9223372036854775807</code>

```
@const
flags = 0b0001 | 0b0010 # 3
@const
masked = flags & 0b0011  # 3
```

```
@const
shifted = 1 << 8      # 256
```

三元條件運算子

```
@const
x = condition ? true_value : false_value
```

對 `condition` 進行求值。若為真，回傳 `true_value`；否則回傳 `false_value`。兩個分支必須具有相同的型別。右結合，因此巢狀三元運算子從右向左結合。

```
@const
x = 3 > 2 ? 10 : 20    # 10
@const
s = false ? "yes" : "no" # "no"

# 巢狀（右結合）
@const
y = true ? (false ? 1 : 2) : 3  # 2
```

範圍運算子

`..` 運算子建立包含兩端的整數範圍。

```
@const
xs = 1 .. 5    # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)    # 1 2 3
```

結果是包含從左運算元到右運算元（兩端皆含）的所有整數的 `List<int>`。

空值合併運算子 (??)

```
@const
x = option_val ?? default_val
```

如果 `option_val` 為 `Some(v)`，則回傳 `v`。否則回傳 `default_val`。右運算元必須與 `Option` 的內部型別相同。

```
@const
a: int? = Some(10)
@const
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0
```

複合賦值運算子

更新變數的簡寫形式。`x op= y` 等價於 `x = x op y`。

運算子	等價的運算式
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>
<code>x %= y</code>	<code>x = x % y</code>
<code>x //= y</code>	<code>x = x // y</code>
<code>x **= y</code>	<code>x = x ** y</code>
<code>x &= y</code>	<code>x = x & y</code>
<code>x = y</code>	<code>x = x y</code>
<code>x ^= y</code>	<code>x = x ^ y</code>
<code>x <<= y</code>	<code>x = x << y</code>
<code>x >>= y</code>	<code>x = x >> y</code>

```
x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x &= 6    # x = 0
```

遞增／遞減運算子

用於將變數增減 1 的後綴運算子。僅可作為陳述式使用。內部分別被轉換為 `x = x + 1` 和 `x = x - 1`。

運算子	等價的運算式
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

```
count = 0
count++    # count = 1
count++    # count = 2
count--    # count = 1

f = 1.5
f++        # f = 2.5 (int 1 會提升為 float)
```

注意：`++` / `--` 只能作為陳述式使用，不能在運算式中使用。`@const` 變數不能使用遞增／遞減（不可變性會被強制執行）。

運算的型別規則

運算	左運算元型別	右運算元型別	結果型別
<code>+</code>	int	int	int
<code>-</code>	float	int / float	float
<code>*</code>	int	float	float
<code>/</code>	任意數值	任意數值	float
<code>//</code>	任意數值	任意數值	int
<code>**</code>	任意數值	任意數值	float
<code>%</code>	int	int	int

運算	左運算元型別	右運算元型別	結果型別
%	float 或 int（其中一方為 float）	—	float
+	str	str	str
== != < <= > >=	數值 / bool / str	同型別	bool
*	str	int	str
in	任意	Set / List / Map<K, V>	bool
not in	任意	Set / List / Map<K, V>	bool
& \ ^ ~ << >> >>>	int	int	int
and or not	bool	bool	bool

運算子多載

可以為使用者定義型別定義運算子的行為。

語法

```
# 二元運算子（2 個引數）
fn operator+(a: MyType, b: MyType) -> MyType:
    ...

# 一元運算子（1 個引數）
fn operator-(a: MyType) -> MyType:
    ...
```

可多載的運算子一覽

種類	運算子
算術（二元）	+ - * / % ** //
比較（二元）	== != < <= > >=
位元（二元）	& \ ^ ~ << >> >>>
邏輯（二元）	and or
一元	- ~ not

二元 / 一元的區別

依引數個數區分。

```
# 二元 -
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# 一元 -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

[English](#) | [日本語](#) | [繁體中文](#)

套件參考

概述

Ry 使用套件系統來組織程式碼。套件可以是單一的 .ry 檔案，或是包含多個 .ry 檔案的目錄。使用 from 陳述式匯入套件。

std 套件（標準函式庫）會自動匯入到每個程式中。

匯入語法

匯入全部定義

```
from math
```

匯入套件內的所有函式與型別。

選擇性匯入

```
from math import add
```

僅匯入指定的定義。

多重選擇性匯入

```
from math import add, sub
```

以逗號分隔選擇性匯入多個定義。

套件解析

以點號分隔的套件名稱按以下方式解析：

匯入陳述式	解析結果
from math	math/ 目錄（套件）或 math.py 檔案
from utils.math	utils/math/ 目錄或 utils/math.py 檔案
from std.str	std/str/ 目錄或 std/str.py 檔案

解析順序

對於每個搜尋路徑：1. 目錄（{path}/）—若存在，載入目錄內的所有 .py 檔案（套件）2. 檔案（{path}.py）—單一檔案（向後相容）

目錄套件

當套件解析為目錄時：- 目錄內的所有 .py 檔案會自動載入 - 以_開頭的檔案會被排除 - 不需要特殊的入口檔案（如__init__.py）- 目錄內檔案中定義的所有函式與型別都會被匯出

私有符號

名稱以 _（底線）開頭的定義為套件內部的私有符號，無法被匯入：

- 萬用匯入（from pkg）會自動排除 _ 前綴的符號
- 具名匯入（from pkg import _helper）會產生編譯錯誤

```
# mylib/internal.py
fn _helper() -> int:      # 私有 — 無法匯入
    return 42
fn public_api() -> int:   # 公開 — 可匯入
    return _helper()
```

```

mypackage/
  math.ry      # fn add(), fn sub()
  string.ry    # fn concat()

from mypackage      # 匯入 add, sub, concat
from mypackage import add  # 僅匯入 add

```

標準函式庫 (std)

std 套件會自動匯入到每個程式中。提供的功能：- 內建函式 (print, len, range 等) - 字串函式 (contains, find, replace 等) - 型別轉換函式 (to_int, to_float, to_str) - 集合函式 (map, filter, sort 等)

子套件

以下子套件需要明確匯入：

套件	說明
<code>std.math</code>	數學常數與函式
<code>std.io</code>	檔案 I/O、標準輸入、位元組轉換

```
from std.math import sqrt, PI, sin
```

也可以從 std 明確匯入特定的定義：

```
from std.str import contains
```

RY_HOME

標準函式庫安裝於 \$RY_HOME/lib/std/。RY_HOME 的預設值為 ~/.ry。

```
export RY_HOME="$HOME/.ry" # 預設值
```

搜尋路徑的優先順序

1. 匯入來源檔案所在的目錄
2. \$RY_HOME/lib (標準函式庫位置)
3. 執行檔相對的 lib/ 目錄
4. RY_PATH 環境變數中包含的路徑 (以冒號分隔)

RY_PATH 環境變數

在 RY_PATH 中以冒號分隔指定目錄，即可新增至套件搜尋路徑。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

限制

限制	詳細
可使用的位位置	僅限頂層 (函式或區塊內不可)
重複匯入	自動跳過 (不會產生錯誤)

限制	詳細
循環匯入	編譯錯誤

```
# 錯誤範例：在區塊內匯入
fn main():
    from math    # 錯誤：僅能在頂層匯入

# OK：多次匯入相同套件不會產生錯誤
from math
from math    # 被跳過
```

建立套件檔案

單一檔案套件

```
# math.ry
fn add(a: int, b: int) -> int:
    return a + b

fn sub(a: int, b: int) -> int:
    return a - b

# main.ry
from math import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

目錄套件

```
mylib/
  math.ry
  string.ry

# main.ry
from mylib import add, concat
```

[English](#) | [日本語](#) | [繁體中文](#)

專案管理

ry init - 專案初始化

將當前目錄初始化為 Ry 專案。

```
ry init
```

生成的檔案與目錄

```
my-project/
  ry.toml          # 專案設定檔
  src/
    main.ry        # 進入點（範例程式碼）
```

行為

1. 若 `ry.toml` 已存在則錯誤結束
2. 建立 `src/` 目錄（若不存在）
3. 生成 `ry.toml`（`name` 為當前目錄名稱）
4. 生成 `src/main.ry`（若已存在則跳過）

ry self-update - 自我更新

將 `ry` 本身更新至最新版本。從 GitHub Releases 下載二進位檔並取代目前的執行檔。

```
ry self-update           # 更新至最新穩定版
ry self-update --nightly # 更新至最新 nightly 預先發行版
ry self-update v0.0.1    # 更新至指定版本
```

行為

1. 顯示目前版本
2. 根據引數解析更新目標版本
 - 無引數：GitHub 的最新穩定發行版（`/releases/latest`）
 - `--nightly`：最新的預先發行版
 - 指定版本：指定標籤的發行版
3. 若與目前版本相同，則以 "Already up to date." 結束
4. 下載二進位檔並取代目前的執行檔

注意事項

- 執行需要 `curl` 和 `tar` 指令
- 若因權限不足導致二進位檔取代失敗，會顯示建議使用 `sudo` 的訊息（不會自動執行 `sudo`）
- 下載會先在臨時目錄中進行；但若跨檔案系統的 `cp` 備援操作被中斷，目標二進位檔可能處於不完整狀態

ry.toml 設定檔

以 TOML 格式記述專案的中繼資料與路徑設定。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"
```

```
[paths]
src = "src"
```

[project] 區段

鍵	說明
<code>name</code>	專案名稱（初始化時為目錄名稱）
<code>version</code>	版本字串
<code>entry</code>	作為進入點的原始碼檔案

[paths] 區段

鍵	說明
src	原始碼目錄

TOML 子集規格

ry.toml 支援以下 TOML 子集。

- 區段標頭：[section]
- 鍵值對：key = "value"（僅字串值）
- 註解：從 # 到行尾
- 空行會被忽略

[English](#) | [日本語](#) | [繁體中文](#)

正規表達式函式參考手冊

正規表達式函式的一覽表。所有函式皆支援 UFCS 記法。模式字串使用標準的正規表達式語法。

注意：Phase 1 中模式以普通字串傳遞。專用的正規表達式字面量語法可能在未來版本中加入。

函式一覽

函式	簽名	說明
regex_match	(str, str) -> bool	文字全體是否匹配模式
regex_search	(str, str) -> int	傳回第一個匹配的起始位置（找不到時傳回 -1）
regex_replace	(str, str, str) -> str	將匹配的部分替換為替換字串
regex_split	(str, str) -> List<str>	以模式匹配進行分割
regex_find_all	(str, str) -> List<str>	傳回所有不重疊的匹配結果

支援的模式語法

語法	說明	範例
abc	字面字元	"hello"
.	任意一個字元（換行除外）	"a.c" 匹配 "abc"、"aXc"
	選擇	"cat dog" 匹配 "cat" 或 "dog"
*	零次或多次重複	"a*" 匹配 ""、"a"、"aaa"
+	一次或多次重複	"a+" 匹配 "a"、"aaa"
?	零次或一次	"a?" 匹配 "" 或 "a"
{n}	恰好 n 次	"a{3}" 匹配 "aaa"
{n,m}	n 到 m 次	"a{2,4}" 匹配 "aa" 到 "aaaa"
{n,}	至少 n 次	"a{2,}" 匹配 "aa"、"aaa"、...
?	零次或多次（非貪婪）	".?" 匹配最短
+?	一次或多次（非貪婪）	".+?" 匹配最短
??	零次或一次（非貪婪）	"a??" 優先零次
{n,m}?	範圍（非貪婪）	"a{2,4}?" 優先 n 次
(...)	群組	"(ab)+" 匹配 "abab"
[abc]	字元類別	"[aeiou]" 匹配母音
[a-z]	字元範圍	"[a-z]+" 匹配小寫單字

語法	說明	範例
[^...]	否定字元類別	"[^0-9]" 匹配非數字
^	字串開頭錨點	"^hello"
\$	字串結尾錨點	"world\$"
\d	數字 [0-9]	"\d+" 匹配數字
\D	非數字 [^0-9]	
\w	單字字元 [a-zA-Z0-9_]	"\w+" 匹配識別符
\W	非單字字元	
\s	空白字元	"\s+" 匹配空格與 Tab
\S	非空白字元	
\b	單字邊界	"\bword\b" 匹配完整單字
\B	非單字邊界	"\Bword" 匹配單字內部
(?i)	忽略大小寫旗標	"(?i)hello" 匹配"HELLO"
\.	跳脫特殊字元	"\." 匹配字面的 .

使用範例

regex_match

```
print(regex_match("[a-z]+", "hello")) # true
print(regex_match("[0-9]+", "hello")) # false
```

regex_search

```
@const
pos = regex_search("[0-9]+", "abc123def")
print(pos) # 3
```

regex_replace

```
@const
s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s) # aXbXcX
```

regex_split

```
@const
parts = regex_split("\\s+", "hello world foo")
print(len(parts)) # 3
print(parts[0]) # hello
```

regex_find_all

```
@const
matches = regex_find_all("[0-9]+", "a1b23c456")
print(len(matches)) # 3
print(matches[0]) # 1
print(matches[1]) # 23
```

範圍量詞

```
print(regex_match("\\d{3}-\\d{4}", "123-4567")) # true
print(regex_match("a{2,4}", "aaa")) # true
print(regex_match("(ab){2,}", "ababab")) # true
```

非貪婪（最短）匹配

```
# 貪婪：匹配最長
@const
g = regex_replace("\.*\\", "\\a\\ and \\b\\", "X")
print(g) # X

# 非貪婪：匹配最短
@const
l = regex_replace("\.*?\\", "\\a\\ and \\b\\", "X")
print(l) # X and X

# 取得個別 HTML 標籤
@const
tags = regex_find_all("<.*?>", "<a> <bb> <ccc>")
print(len(tags)) # 3
```

注意：非貪婪匹配控制整體匹配長度。沒有使用括號分組時，greedy/lazy 混合模式可能與 PCRE 引擎行為不同。

單字邊界

```
# 匹配完整單字
@const
pos = regex_search("\\bworld\\b", "hello world")
print(pos) # 6

# 取得所有單字
@const
words = regex_find_all("\\b\\w+\\b", "hello world foo")
print(len(words)) # 3

# \\B 匹配非邊界（單字內部）
@const
pos2 = regex_search("\\Bworld", "helloworld")
print(pos2) # 5
```

忽略大小寫匹配

```
# (?i) 放在模式開頭即可忽略大小寫
print(regex_match("(?i)hello", "HELLO")) # true
print(regex_match("(?i)hello", "Hello")) # true

# 字元類別也適用
print(regex_match("(?i)[a-z]+", "ABC")) # true

# replace 和 find_all 也可使用
@const
s = regex_replace("(?i)hello", "Hello HELLO hello", "X")
print(s) # X X X
```

注意：(?i) 必須出現在模式的開頭，並適用於整個模式。不支援部分忽略大小寫（例如 (?i:sub)pattern）。

UFCS 記法

```
# pattern.function(text, ...)
@const
m = "[a-z]+".regex_match("hello")
print(m) # true
```

[English](#) | [日本語](#) | [繁體中文](#)

結構體參考

概述

結構體是堆疊上的值型別。使用 `record` 關鍵字定義。結構體可以使用 `invariant` 子句定義不變量。參閱[契約式設計](#)。

命名慣例：結構體名稱必須使用 PascalCase (如 `Point`、`Rectangle`)。欄位名稱必須使用 snake_case。編譯器會強制執行這些慣例。

定義語法

`record` 型別名:
 欄位名: 型別
 欄位名: 型別

範例

```
record Point:
  x: int
  y: int

record Rectangle:
  width: float
  height: float
```

建構子

按照欄位定義順序傳遞引數。不支援具名引數。

```
@const
p = Point(10, 20)
@const
r = Rectangle(3.0, 4.5)
```

欄位存取

使用點記法讀取欄位。

```
@const
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

欄位賦值

變數宣告	欄位賦值
可變 (無 <code>@const</code>)	可以
<code>@const</code>	編譯錯誤

```
p = Point(10, 20)
p.x = 100      # OK: 可變變數

@const
q = Point(10, 20)
q.x = 100      # 錯誤: @const 變數的欄位不可變更
```

作為函式引數與回傳值使用

```
fn distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

fn make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

巢狀結構體

可以將結構體作為另一個結構體的欄位使用。

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

@const
c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

限制與錯誤

限制	詳細
相同欄位名重複	編譯錯誤
<code>@const</code> 變數的欄位賦值	編譯錯誤
直接將結構體傳給 <code>print</code>	編譯錯誤 (print 不支援)

```
# 錯誤範例: 相同欄位名重複
record Bad:
```

```

x: int
x: int    # 錯誤

# 錯誤範例：將結構體傳給 print
@const
p = Point(1, 2)
print(p)    # 錯誤

```

列舉型別 (enum)

概述

列舉型別是具名常數的集合。內部以 i64 整數 (0, 1, 2, ...) 表示。

定義語法

```

enum 型別名:
    變體名
    變體名
    ...

```

範例

```

enum Color:
    Red
    Green
    Blue

```

變體存取

使用 `::` 運算子存取變體。

```

@const
c = Color::Red
print(c)    # Red

```

比較

enum 值為整數，因此可以直接使用 `==` / `!=` 進行比較。

```

print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true

```

在 if 陳述式中使用

```

@const
c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
    print("green")
else:
    print("blue")

```

函式引數

型別名稱使用 `enum` 名稱。

```
fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

使用 `print()` 會輸出變體名稱。

```
@const
c = Color::Blue
print(c)    # Blue
```

限制與錯誤

限制	詳細
變體存取為 <code>EnumName::VariantName</code>	必須使用 <code>::</code> 運算子
變體值為自動分配	0, 1, 2, ... 的連續編號（無法手動指定）
比較為整數比較	可使用 <code>==</code> 、 <code>!=</code>

[English](#) | [日本語](#) | [繁體中文](#)

測試功能

Ry 內建 RSpec 風格的測試語法。使用 `ry test` 子指令執行測試檔案。

執行方式

```
ry test          # 自動探索並執行專案內所有 *.test.ry 檔案
ry test tests/spec  # 遞迴執行指定目錄下所有 *.test.ry 檔案
ry test test_file.ry # 執行指定的測試檔案
```

結束代碼為 0 表示所有測試通過，1 表示有測試失敗。

自動探索模式

不帶引數執行 `ry test` 時：

1. 搜尋 `ry.toml` 以找到專案根目錄
2. 在專案根目錄下遞迴探索所有 `*.test.ry` 檔案（`.git`、`build`、`node_modules` 會被跳過）
3. 逐一執行並彙總結果

語法

describe / it

```
describe("說明文字", fn():
    it("測試案例名稱", fn():
```

```

    # 測試主體
    expect(實際值).to_eq(預期值)
  )
)

```

- `describe` 和 `it` 接受描述字串和 `lambda` 引數 `fn()`：作為第二個參數
- `describe` 區塊內可以撰寫 `it` 區塊及其他語句（如變數宣告等）
- 各 `it` 區塊為獨立的測試案例
- `describe / expect` 僅能在 `ry test` 中使用（在一般的 `ry` 執行中會產生編譯錯誤）

尾隨區塊語法

任何函式呼叫（`describe/it/mock` 除外）都可以使用尾隨區塊語法。在 `()` 後加上 `:` 會將縮排區塊作為無參數 `lambda` 傳入最後的參數位置：

```

# 以下兩者等價：
foo("arg"):
  bar()

foo("arg", fn():
  bar()
)

```

`expect` / 匹配器

匹配器	說明	支援型別
<code>to_eq(expected)</code>	相等比較	int, float, bool, str
<code>to_not_eq(expected)</code>	不相等	int, float, bool, str
<code>to_be_true()</code>	為 <code>true</code>	bool
<code>to_be_false()</code>	為 <code>false</code>	bool
<code>to_be_none()</code>	為 <code>None</code>	Option
<code>to_be_some()</code>	Option 為 <code>Some</code>	Option
<code>to_contain(val)</code>	容器包含值	List, Set, Map, str
<code>to_not_contain(val)</code>	容器不包含值	List, Set, Map, str
<code>to_be_greater_than(v)</code>	<code>actual > v</code>	int, float
<code>to_be_less_than(v)</code>	<code>actual < v</code>	int, float
<code>to_be_greater_than_or_eq(v)</code>	<code>actual >= v</code>	int, float
<code>to_be_less_than_or_eq(v)</code>	<code>actual <= v</code>	int, float
<code>to_have_length(n)</code>	長度為 <code>n</code>	List, Set, Map, str
<code>to_be_empty()</code>	長度為 0	List, Set, Map, str
<code>to_start_with(prefix)</code>	字串以 <code>prefix</code> 開頭	str
<code>to_end_with(suffix)</code>	字串以 <code>suffix</code> 結尾	str

輸出格式

```

Calculator
+ adds numbers
+ subtracts
- fails test (紅色)
  line 10: expected 3, got 2

```

2 passed, 1 failed

- + 為成功（綠色），- 為失敗（紅色）
 - 失敗時顯示行號與預期值/實際值
-

範例

```
describe("Arithmetic", fn():
  it("adds integers", fn():
    expect(1 + 2).to_eq(3)

  )
  it("compares strings", fn():
    expect("hello").to_eq("hello")

  )
  it("checks booleans", fn():
    expect(3 > 1).to_be_true()

  )
)
describe("Booleans", fn():
  it("false check", fn():
    expect(1 > 2).to_be_false()

  )
)
```

模擬 (Mock)

mock(fn_name, replacement)

在當前 it 區塊中將函式替換為模擬實作。it 區塊結束時模擬會自動清除。

```
fn fetch_data() -> str:
  return "real data"

describe("mocking", fn():
  it("replaces function", fn():
    mock(fetch_data, fn(): "fake")
    expect(fetch_data()).to_eq("fake")

  )
  it("auto-restores", fn():
    expect(fetch_data()).to_eq("real data")

  )
)
```

- 第一個參數為函式名稱（識別符，非字串）
- 第二個參數為替換用 lambda
- 替換函式必須與原始函式具有相同的參數型別和回傳型別
- it 區塊結束時模擬會自動還原

verify(fn_name)

回傳模擬函式被呼叫的次數 (int)。

```
describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn(): "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

模擬的限制事項

- 不支援多載函式的模擬
- 不支援使用捕獲閉包進行模擬（僅支援純 lambda）
- 不支援 @native fn 的模擬

參數化測試 (@each)

@each 可以用多組參數執行同一個測試。將元組列表附加到 it 區塊：

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

- 列表必須包含與 lambda 參數數量相同的元組
- 描述中的 {0}, {1}, ... 會被替換為參數值
- 每個元組生成一個獨立的測試案例
- 支援的參數型別: int, float, bool, str

基於屬性的測試 (@property)

@property 生成隨機輸入並多次執行測試：

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- count=N 指定隨機試驗次數（預設: 100）
- 失敗時會顯示反例（導致失敗的輸入值）
- 在第一次失敗時停止測試
- 支援的參數型別: int ([-1000, 1000])、float ([-1000.0, 1000.0])、bool、str (隨機 ASCII、0-20 字元)

限制事項

- 不支援 describe 的巢狀
- 不支援 before_each / after_each

[English](#) | [日本語](#) | [繁體中文](#)

型別參考

型別一覽

型別	內部表示	字面值範例	說明
int	i64	42, -7, 0xFF, 0b1010	64 位元有號整數
byte	i8	(無專用字面值)	無號 8 位元整數 (0-255)。透過型別標註 b: byte = 42 使用
float	f64	3.14, 0.5	64 位元浮點數
bool	!l	true, false	布林值
str	ptr	"hello", "", "a\nb"	字串 (堆積上的不可變位 元組序列)
Unit	void	(無回傳值)	省略回傳值型別時的隱式 回傳型別
Option<T> (T1, T2, ...)	{ i1, T } LLVM StructType (literal)	Some(42), None (1, 3.14)	可能存在值的型別 元組型別
List<T>	ptr (堆積)	[1, 2, 3]	動態陣列
Map<K, V>	ptr (堆積)	{"a": 1}	雜湊映射
Set<T>	ptr (堆積)	{1, 2, 3}	不重複的集合
fn(T1, T2) -> R 使用者定義型別	ptr (函式指標) LLVM StructType (named)	fn(x: int): x * 2 record Point: ...	函式型別 以 record 關鍵字定義的 結構體
enum	i64 / 標籤聯合	Color::Red, Shape::Circle(3.14)	以 enum 關鍵字定義的列 舉型別 (支援關聯資料)
Error	{ ptr, i64 }	Error("msg"), Error("msg", 404)	內建錯誤型別
T1 \ T2	{ i64, [N x i8] }	int \ str	union 型別 (可持有多種 型別之一)
int 字面量	i64	42, 0 \ 1	int 字面量型別 (值限制)
str 字面量	ptr	"N" \ "S"	str 字面量型別 (值限制)
範圍	i64	1..12, -10..10	範圍型別 (包含兩端的整 數範圍限制)

型別標註語法

宣告變數時可以明確指定型別。當型別可推論時可以省略。

```
@const
x: int = 42
@const
b: byte = 255
@const
f: float = 3.14
@const
s: str = "hello"
@const
b: bool = true
@const
opt: Option<int> = Some(10)
@const
t: (int, float) = (1, 3.14)
```

```

@const
xs: List<int> = [1, 2, 3]
@const
m: Map<str, int> = {"a": 1}
@const
s: Set<int> = {1, 2, 3}
@const
fn_val: fn(int) -> int = fn(x: int): x * 2
@const
u: int | str = 42

```

可用型別名稱一覽

型別名稱	備註
int	內建純量型別
byte	內建純量型別（無號 0-255）
float	內建純量型別
bool	內建純量型別
str	內建字串型別
Unit	無回傳值函式的回傳型別
Option<T>	泛型型別（T 為任意型別）
(T1, T2, ...)	元組型別（元素數量和型別組合任意）
List<T>	泛型動態陣列型別
Map<K, V>	泛型雜湊映射型別
Set<T>	泛型集合型別
fn(T1, ...) -> R	函式型別
Error	內建錯誤型別（message: str、code: int）
T1 \ T2 \ ...	union 型別（以 \ 分隔的多個型別之一）
使用者定義型別名稱	以 record 或 enum 關鍵字宣告的型別
int 字面量型別	以 int 字面量值限制（例：42、0 \ 1）
str 字面量型別	以字串字面量值限制（例："N" \ "S"）
範圍型別	以整數範圍限制（例：1..12、-10..10）

型別別名

type 關鍵字為現有型別建立新的名稱。別名與原始型別完全互換。

```

type Meters = float
type StringList = List<str>

```

```

@const
d: Meters = 3.14
@const
names: StringList = ["Alice", "Bob"]

```

命名慣例：型別別名名稱必須使用 PascalCase（如 Meters、StringList）。編譯器會強制執行此慣例。

型別別名也可以用於函式型別、字面量型別和範圍型別：

```

type Callback = fn(int, int) -> int

@const
add: Callback = fn(a: int, b: int): a + b
print(add(3, 4))    # 7

```

```

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

@const
m: Month = 6
@const
d: Direction = "N"
@const
n: Digit = 5

```

字面量型別

字面量型別將變數的值限制為特定的常數值。對於常數值，在編譯時進行約束檢查；對於動態值，在執行時進行約束檢查。

int 字面量型別

```

@const
x: 42 = 42          # 單一字面量型別
@const
y: 0 | 1 = 0        # int 字面量的 union
z: 0 | 1 = 0
z = 1               # OK
# z = 2             # 編譯錯誤（常數）或執行時錯誤（動態值）

```

str 字面量型別

```

@const
dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # 編譯錯誤

```

約束檢查

- 編譯時：當賦值為常數（ConstantInt 或字串字面量）時，在編譯時檢查，違反時產生編譯錯誤
 - 執行時：當值為動態（如函式回傳值）時，在執行時檢查，違反時程式以錯誤退出
-

範圍型別

範圍型別將整數變數的值限制在連續的範圍內（包含兩端）。

```

@const
month: 1..12 = 6      # OK
# @const bad: 1..12 = 0    # 編譯錯誤：超出範圍
# @const bad: 1..12 = 13   # 編譯錯誤：超出範圍

@const
t: -10..10 = -5       # 支援負數範圍

```

使用可變變數重新賦值（執行時檢查）

```

x: 1..12 = 6
x = 12          # OK

```

```
# x = dynamic_value()          # 執行時檢查：超出範圍則錯誤退出
```

在函式參數中使用

```
fn set_month(m: 1..12) -> int:
    return m
```

```
set_month(6)          # OK
# set_month(13)       # 編譯錯誤（常數引數）
```

none 關鍵字與 Option 型別簡寫

none 關鍵字表示 Option 型別的值不存在，等同於 None。

T? 語法是 Option<T> 的簡寫。

```
@const
x: int? = 42          # 等同於 Option<int>
@const
y: int? = none        # 等同於 None

fn find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

F-String（字串插值）

使用 f"..." 語法進行字串插值。{} 內的表達式會被求值並轉換為字串。

```
@const
name = "world"
print(f"Hello {name}")    # Hello world

@const
a = 1
@const
b = 2
print(f"{a} + {b} = {a + b}")  # 1 + 2 = 3
```

插值中支援的型別

{ } 內可使用求值結果為 int、float、bool 或 str 的任意表達式。

跳脫序列

序列	輸出
{{	{（字面大括號）
}}	}（字面大括號）
\n \r \t \\ \"	與普通字串相同

```
print(f"{{{braces}}}")    # {braces}
```

型別轉換 (as)

使用 `as` 關鍵字進行明確的型別轉換。

```
@const
x = 42 as float      # 42.0
@const
y = 3.14 as int      # 3
@const
z = 1 as bool        # true
@const
s = 42 as str         # "42"
@const
b = 255 as byte      # byte 值 255
```

支援的轉換

來源	目標	行為
int	float	SIToFP
float	int	截斷 (FPToSI)
int	bool	0 → false、非零 → true
bool	int	false → 0、true → 1
int / float / bool	str	字串表示
int	byte	截斷 (低 8 位元)
byte	int	零擴展

不支援的轉換 (例如 `str as int`) 會產生編譯錯誤。字串轉數值請使用 `to_int()` / `to_float()`。

帶關聯資料的 enum (ADT)

在變體名稱後面加上括號並指定型別，enum 變體就可以攜帶關聯資料。不帶括號的變體仍然是單純的標籤。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

建構子

使用 `EnumName::Variant(value)` 語法建立帶有資料的變體。

```
@const
c = Shape::Circle(3.14)
@const
r = Shape::Rectangle(4.0, 5.0)
@const
p = Shape::Point
```

帶綁定的模式匹配

使用 `case EnumName::Variant(binding):` 形式取出關聯資料。

```
match c:
    case Shape::Circle(r):
        print(r)          # 3.14
    case Shape::Rectangle(w, h):
```

```

    print(w)
    print(h)
case Shape::Point:
    print("point")

```

內部表示

ADT enum 以標籤聯合的形式儲存：`{ i64 tag, [N x i8] data }`，`N` 的大小足以容納最大變體的酬載。

泛型 enum

enum 可以使用角括號語法 `<T>` 帶有序別參數，使相同的 enum 結構可以持有不同型別的酬載。

```

enum MyOption<T>:
    MySome(T)
    MyNone

```

使用方式

當編譯器無法推論型別時，需提供具體的型別引數來實例化。

```

@const
a = MyOption<int>::MySome(42)
@const
b = MyOption<int>::MyNone

match a:
    case MyOption::MySome(v):
        print(v)          # 42
    case MyOption::MyNone:
        print("none")

```

Error 型別

用於錯誤處理的內建型別。Error 具有兩個欄位：`message` (str) 和 `code` (int)。

```

@const
e = Error("something went wrong")          # code 預設為 0
@const
e2 = Error("not found", 404)               # 明確指定 code

print(e.message)    # something went wrong
print(e2.code)      # 404
print(e2)           # Error: not found (code: 404)

```

錯誤處理慣例

可能失敗的函式回傳 `(T, Error?)` 元組：

```

fn divide(a: int, b: int) -> (int, Error?):
    if b == 0:
        return (0, Some(Error("division by zero")))
    return (a // b, none)

```

```

@const
val, err = divide(10, 2)
match err:
    case Some(e):
        print(e.message)
    case None:
        print(val)           # 5

```

!! 運算子（錯誤傳播）

!! 後綴運算子從 (T, Error?) 元組中取出值。如果錯誤存在，會將其傳播給外層函式。

```

fn read_file(path: str) -> (str, Error?):
    if path == "":
        return ("", Some(Error("empty path")))
    return ("content", none)

fn process() -> (str, Error?):
    @const
    data = read_file("test.txt")!!    # 如果有錯誤則傳播
    return (data, none)

```

外層函式也必須回傳 (X, Error?) 才能使用 !!。

內部表示

Error 以 { ptr message, i64 code } 表示。

union 型別

可以使用 | 宣告可能持有多種型別的變數。

```

@const
x: int | str = 42
x = "hello"    # 可重新賦值 (union 中的任一型別)
print(x)      # hello

```

在函式引數與回傳值中的使用

```

fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"

```

內部表示

union 型別以 { i64 tag, [N x i8] data } 表示。tag 表示各組成型別的索引（按字母順序排序後），data 是最大組成型別大小的位元組陣列。

限制

- 賦值不屬於 union 的型別會產生編譯錯誤
- int | str 和 str | int 是相同的型別（會被正規化）

- 使用 `print()` 輸出 union 值時，會根據執行時的 tag 以適當的型別顯示

型別規則（運算時的型別轉換）

運算	左運算元	右運算元	結果型別	備註
<code>+</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>-</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>*</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>+</code>	<code>byte</code>	<code>byte</code> 或 <code>int</code>	<code>int</code>	<code>byte</code> 在運算時以 ZExt 提升為 <code>int</code>
<code>-</code>	<code>byte</code>	<code>byte</code> 或 <code>int</code>	<code>int</code>	隱式 <code>float</code> 提升
<code>*</code>	<code>float</code> 或 <code>int</code>	<code>float</code> 或 <code>int</code> （其中一方為 <code>float</code> ）	<code>float</code>	
<code>/</code>	任意數值	任意數值	<code>float</code>	始終為 <code>float</code>
<code>//</code>	任意數值	任意數值	<code>int</code>	<code>float</code> 輸入會截斷轉換
<code>**</code>	任意數值	任意數值	<code>float</code>	使用 <code>libm pow</code>
<code>%</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>%</code>	<code>float</code> 或 <code>int</code>	<code>float</code> 或 <code>int</code> （其中一方為 <code>float</code> ）	<code>float</code>	
<code>+</code>	<code>str</code>	<code>str</code>	<code>str</code>	字串串接
<code>==</code>	<code>str</code>	<code>str</code>	<code>bool</code>	字典序比較
<code>!=</code>	<code>str</code>	<code>str</code>	<code>bool</code>	
<code><=</code>	<code>str</code>	<code>str</code>	<code>bool</code>	
<code>>=</code>	<code>str</code>	<code>str</code>	<code>bool</code>	
<code>==</code>	數值或 <code>bool</code>	數值或 <code>bool</code>	<code>bool</code>	
<code>!=</code>	數值或 <code>bool</code>	數值或 <code>bool</code>	<code>bool</code>	
<code><=</code>	數值或 <code>bool</code>	數值或 <code>bool</code>	<code>bool</code>	
<code>>=</code>	數值或 <code>bool</code>	數值或 <code>bool</code>	<code>bool</code>	
<code>in</code>	任意	Set	<code>bool</code>	元素是否包含在集合中
<code>&</code>	<code>int</code>	<code>int</code>	<code>int</code>	對 <code>float</code> 會產生錯誤
<code> </code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>^</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>~</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code><<</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>>></code>	<code>int</code>	<code>int</code>	<code>int</code>	

跳脫序列（`str` 字面值內）

序列	意義
<code>\n</code>	換行
<code>\r</code>	回車
<code>\t</code>	定位字元
<code>\\</code>	反斜線
<code>\"</code>	雙引號
<code>\0</code>	空字元

型別安全性限制

- 沒有隱式型別轉換 — 混合使用 `int` 和 `float` 時會發生 `float` 提升，但除此之外不存在隱式轉換。`byte` 在運算時會自動提升為 `int` (ZExt)。僅在型別標註 `b: byte = 42` 時允許從 `int` 字面值到 `byte` 的窄化轉換。
- 變數型別在宣告時固定 — 一旦以 `int` 宣告的變數，就無法重新賦值為 `float`。
- 位元運算僅限 `int` — 對 `float` 或 `bool` 使用位元運算會產生編譯錯誤。
- `bool` 以外的型別也可用於條件式 — `if` 的條件式可使用 `int` (`0 = false`、非 `0 = true`) 等 `bool` 以外的型別。